Application Modernization with MTA

Version 1, 2026-11-04

Introduction

Welcome to this quick course on the ***Application Modernization with MTA***. This course is the `fifth` in a series of `six` courses about **Application Platform Delivery**:

- *Introduction to Application Platforms and Platform Engineering*
- *Software Factory Foundations with OpenShift Dev Spaces*
- *Software Delivery with OpenShift Pipelines and GitOps*
- *DevEx with Red Hat Developer Hub*
- *Application Modernization with MTA* (*This course*)
- *Trusted Software Supply Chain (TSSC)* (Coming Soon)

What's in this course?

In this course, you'll learn how to modernize enterprise Java applications using **Red Hat Migration Toolkit for Applications (MTA)**, **Red Hat Developer Lightspeed**, and **Red Hat OpenShift Dev Spaces**. You'll assess and analyze applications, identify migration requirements, and use AI-assisted recommendations to modernize an application from **JBoss EAP 7 to JBoss EAP 8**.

Duration: 1.5 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- Andrew Block (SME)
- Bill Wheatley (SME)
- Bryan Saunders (SME)
- Evan Shortiss (SME)
- Julio Villarreal Pelegrino (SME)
- Kristopher Godek (SME)
- Roman Martin Gil (SME)
- Senthil Kumar (SME)
- Sergio Canales Espinoza (SME)
- Yogita Soni (Content Architect)
- Lisa Edgell (Learning Product Manager)
- Caitlyn Rymarchyk (Learning Product Manager)
- Anna Swicegood (Learning Product Manager)

Learning Objectives

By the end of this module, you will be able to:

- Explain how MTA supports enterprise application modernization
- Assess and analyze Java applications for migration to JBoss EAP 8
- Identify Java EE to Jakarta EE migration requirements
- Configure MTA with Red Hat MaaS and Developer Lightspeed
- Use the MTA extension in Red Hat OpenShift Dev Spaces
- Apply AI-assisted migration recommendations to modernize applications
- Validate and build a modernized application

Prerequisites

This course assumes that you have the following prior experience:

1. Red Hat OpenShift Technical Overview (DO080) v4.20 or Kubernetes Basics

Lab Environment

No new lab environment is required for this module. Continue using the **same RHDP lab environment** that you provisioned in ***Software Factory Foundations with OpenShift Dev Spaces***.

The environment includes the **"Service Enablement: App Platform Software Factory"** and **"Service Enablement: App Platform Runtime"** labs required for this module.

Do **not** provision, delete, reset, or reprovision the environment. The labs in this module build on the applications, configurations, and platform resources created in ***Software Factory Foundations with OpenShift Dev Spaces***.

The cluster URLs and credentials are available from the **My Services** page in RHDP.

OpenShift Clusters

Environment	Purpose	Used In
Service Enablement: App Platform Software Factory (Primary Cluster)	Development platform where you'll build the internal developer platform, configure CI/CD, and deploy applications.	• *Software Factory Foundations with OpenShift Dev Spaces* • *Software Delivery with OpenShift Pipelines and GitOps* • *DevEx with Red Hat Developer Hub* • *Application Modernization with MTA*
Service Enablement: App Platform Runtime (Secondary Cluster)	Runtime environment used to demonstrate GitOps-based promotion between environments.	• *Software Delivery with OpenShift Pipelines and GitOps*

The required namespaces have already been created and are ready for use.

Internal Lab Account Credentials

Use the following accounts throughout the learning path.

Username	Password	Purpose
`platform-admin`	`openshift`	Platform administrator
`platform-developer`	`openshift`	Application developer
`platform-user`	`openshift`	Standard user

Unless instructed otherwise, authenticate to GitLab, Quay, Vault, and other integrated services using

Keycloak Single Sign-On (SSO).

Platform Services

The following services are pre-configured and available throughout the labs.

Service	Purpose
Red Hat Build of Keycloak	Provides centralized authentication and Single Sign-On (SSO).
GitLab	Hosts the application source code and Git repositories.
Red Hat Quay	Stores container images built during the labs.
OpenShift GitOps	Continuously synchronizes applications and platform resources from Git.
HashiCorp Vault	Securely stores secrets used by applications and CI/CD pipelines.

All service URLs are available from the **Red Hat Demo Platform (RHDP)** **Lab Information** in **My Services** section page.

MTA Overview



What's in this section?

In this section, you'll learn how to modernize enterprise Java applications using Red Hat Migration Toolkit for Applications (MTA) and Red Hat Developer Lightspeed.

You'll explore how MTA supports application modernization at scale, assess migration requirements, and analyze the Coolstore application for migration from JBoss EAP 7 to JBoss EAP 8. You'll then use AI-assisted recommendations in Red Hat OpenShift Dev Spaces to identify and apply migration changes and validate the modernized application.

Modernizing Applications with MTA

The Challenge

Many organizations rely on large portfolios of applications built on traditional application servers and legacy frameworks.

While these applications continue to provide business value, they often become increasingly difficult to maintain, secure, and deploy on modern cloud-native platforms such as Red Hat OpenShift.

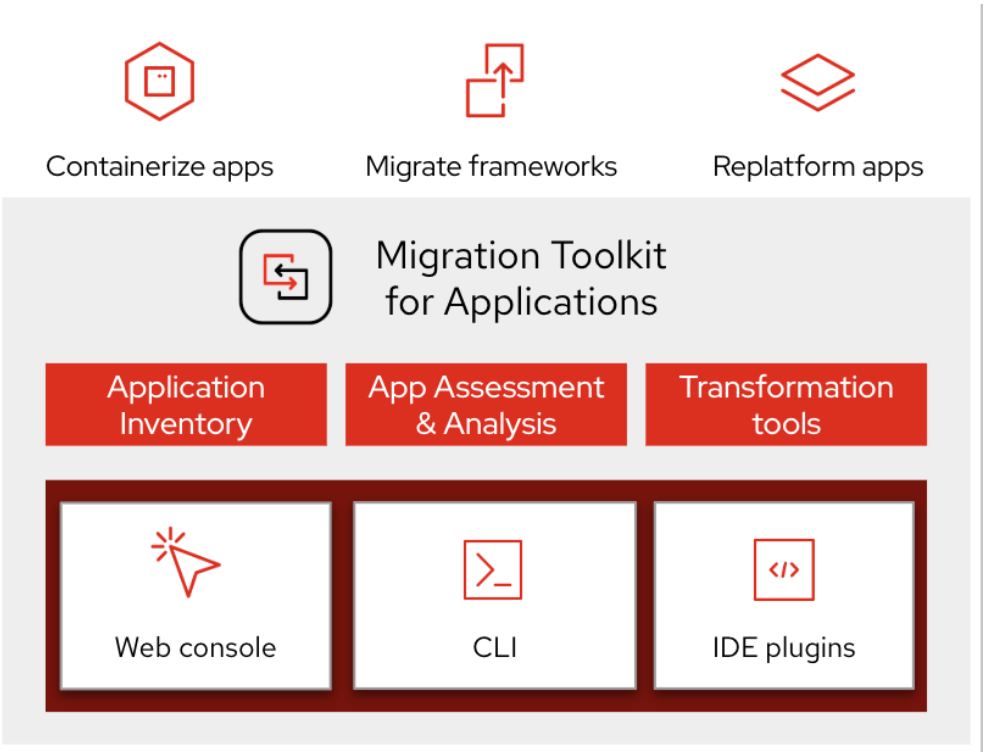


Figure 1. Modernizing Existing Applications (Click the image to view it in full size in a new browser tab)

Development teams are frequently faced with questions such as:

- Which applications should be modernized first?
- How much effort will each migration require?
- Which technologies are no longer supported?
- What code changes are needed before moving to OpenShift?
- Can modernization happen without rewriting the entire application?

Instead of manually reviewing thousands of source files, organizations need an automated way to assess existing applications and plan modernization efforts.



Modernization does not always mean rewriting an application.

Many organizations achieve significant value by incrementally modernizing existing applications while preserving their existing business logic.

What is Migration Toolkit for Applications?

Migration Toolkit for Applications (MTA) helps organizations assess, plan, and modernize existing applications for modern runtimes and cloud-native platforms.

Rather than manually reviewing application source code, MTA analyzes the application, identifies migration issues, estimates modernization effort, and generates detailed reports that guide developers through the modernization process.

This enables modernization projects to become predictable, repeatable, and easier to manage across large application portfolios.

How MTA Helps Modernize Applications

MTA provides several capabilities that simplify application modernization.

Capability	Purpose
Application Analysis	Scans application source code, configuration files, and dependencies to identify migration issues.
Migration Rules	Uses pre-built rule sets to detect changes required for different runtimes, frameworks, and migration targets.
Portfolio Assessment	Analyzes multiple applications to help prioritize modernization efforts across large application portfolios.
Web Console	Provides a centralized interface for managing modernization projects and reviewing migration reports.
CLI	Enables automated application analysis from developer workstations and CI/CD pipelines.

Modernization Strategies

Not every application requires the same modernization approach.

Organizations typically adopt one of the following strategies depending on business requirements.

Strategy	Description
Rehost	Move the application to new infrastructure with little or no code changes.
Replatform	Containerize the application and move it to OpenShift while making minimal application changes.
Refactor	Redesign the application to adopt cloud-native architectures such as microservices or modern Java frameworks like Quarkus.

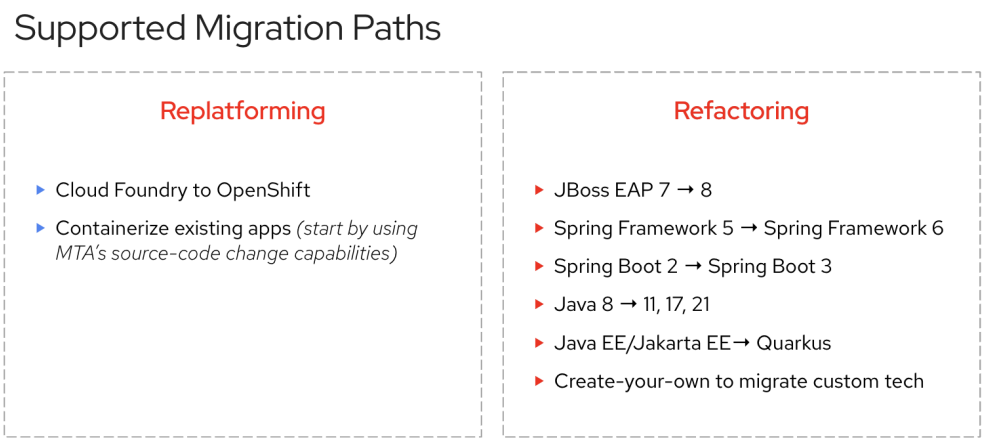


Figure 2. Application Modernization Strategies (Click the image to view it in full size in a new browser tab)

As modernization effort increases, organizations gain greater scalability, flexibility, and cloud-native capabilities.

Supported Modernization Targets

After modernization, applications typically run on one of Red Hat's supported runtimes.

Runtime	Typical Use Case
Red Hat build of Quarkus	Modern Kubernetes-native Java applications with fast startup and low memory consumption.
JBoss Enterprise Application Platform (EAP)	Enterprise Jakarta EE applications requiring long-term support.
Red Hat build of Spring Boot	Modern Spring Boot applications with enterprise support.
Node.js, .NET, and Go	Modernize applications across polyglot development environments.

MTA in the Software Factory

Application modernization becomes another workflow within the Software Factory.

Instead of starting with a new application, developers begin with an existing application and use MTA to prepare it for modern platforms.

A typical modernization workflow includes:

1. Analyze the existing application using MTA.
2. Review migration reports and recommended changes.
3. Modernize the application to use a supported runtime.
4. Commit the updated application to Git.
5. Validate changes through the CI pipeline.
6. Deploy the application using GitOps.
7. Continue operating the application using the same Software Factory processes.

This allows organizations to modernize existing applications while continuing to use the same standardized development, security, and deployment workflows.



Migration Toolkit for Applications does not automatically rewrite applications.

Instead, it analyzes existing applications, identifies migration issues, and provides actionable guidance that helps development teams modernize applications efficiently.

Reflection

▼ *Think about your own organization (Click to expand)*



- Which applications are the best candidates for modernization?
- Would they require rehosting, replatforming, or refactoring?
- How could automated application analysis reduce modernization effort?
- How would integrating MTA into your Software Factory accelerate software delivery?

Key Takeaway

Migration Toolkit for Applications helps organizations modernize existing applications by automatically analyzing source code, identifying migration issues, and providing actionable guidance. This enables development teams to migrate applications to modern platforms while continuing to use the same standardized Software Factory for building, testing, deploying, and operating software.

MTA at Scale

The Challenge

In the previous lesson, you learned how Migration Toolkit for Applications (MTA) analyzes individual applications and recommends an appropriate modernization strategy.

While modernizing a single application is relatively straightforward, enterprise organizations often need to modernize hundreds—or even thousands—of applications across multiple business units and development teams.

This raises several important questions:

- Which applications should be modernized first?
- Which applications depend on each other?
- How much effort will each migration require?
- Which modernization strategy should be used?
- How can multiple teams modernize applications consistently?

Without a standardized approach, migration projects become difficult to prioritize, estimate, and coordinate.



Successful application modernization is not just about migrating individual applications—it requires managing an entire application portfolio with consistent processes, shared knowledge, and repeatable engineering practices.

Scaling Modernization Across the Enterprise

Migration Toolkit for Applications (MTA) provides organizations with a portfolio-level view of their application landscape.

Instead of analyzing applications one at a time, architects and platform teams can evaluate an entire portfolio, identify modernization priorities, estimate migration effort, and coordinate work across multiple teams.

This enables organizations to transform modernization from isolated projects into a repeatable engineering process.

The Application Migration Factory

The **Application Migration Factory** provides a standardized workflow for modernizing applications at scale.

Rather than every migration team inventing its own process, organizations follow a common workflow that combines discovery, assessment, analysis, migration guidance, and shared knowledge.

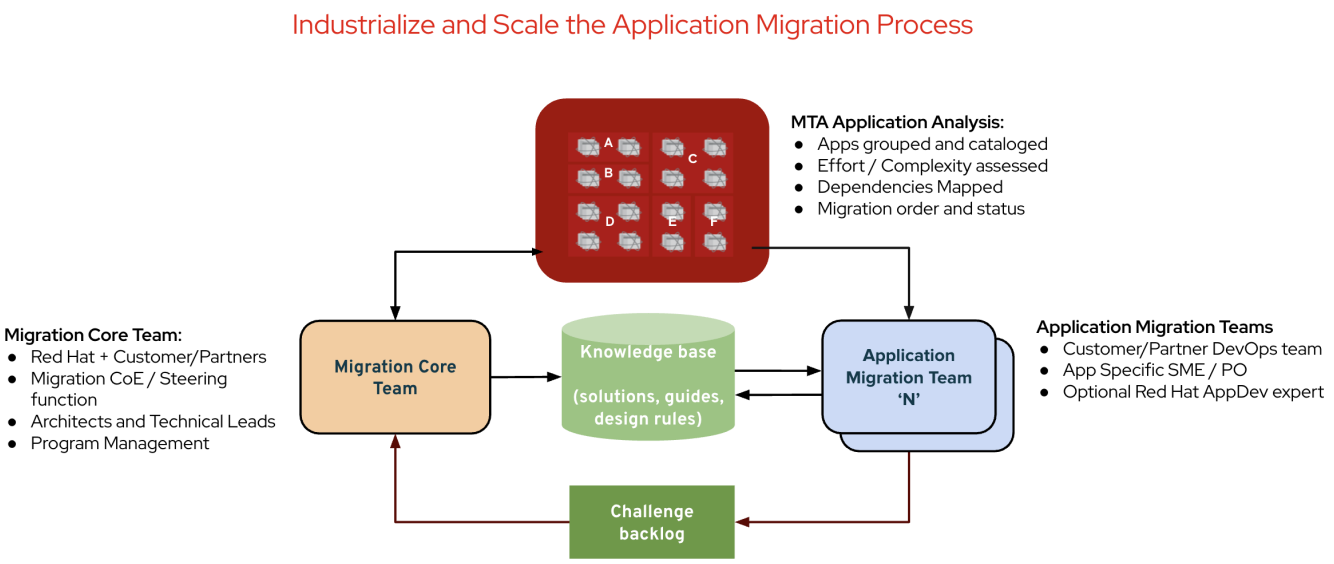


Figure 3. Application Migration Factory (Click the image to view it in full size in a new browser tab)

The workflow is made up of several key stages.

Stage	Purpose
Application Inventory	Maintains a centralized inventory of enterprise applications, including metadata, business domains, technology stacks, and dependencies.
Application Assessment	Evaluates modernization readiness, estimates migration effort, identifies risks, and recommends an appropriate modernization strategy.
Application Analysis	Scans application source code and binaries using migration rules to identify compatibility issues and required code changes.
Knowledge Base	Captures reusable migration guidance, best practices, design decisions, and organization-specific documentation for future projects.

Stage	Purpose
Migration Teams	Allows multiple development teams to modernize applications in parallel while following standardized engineering practices.

Portfolio Assessment

One of MTA's strengths is helping organizations understand their application portfolio before migration begins.

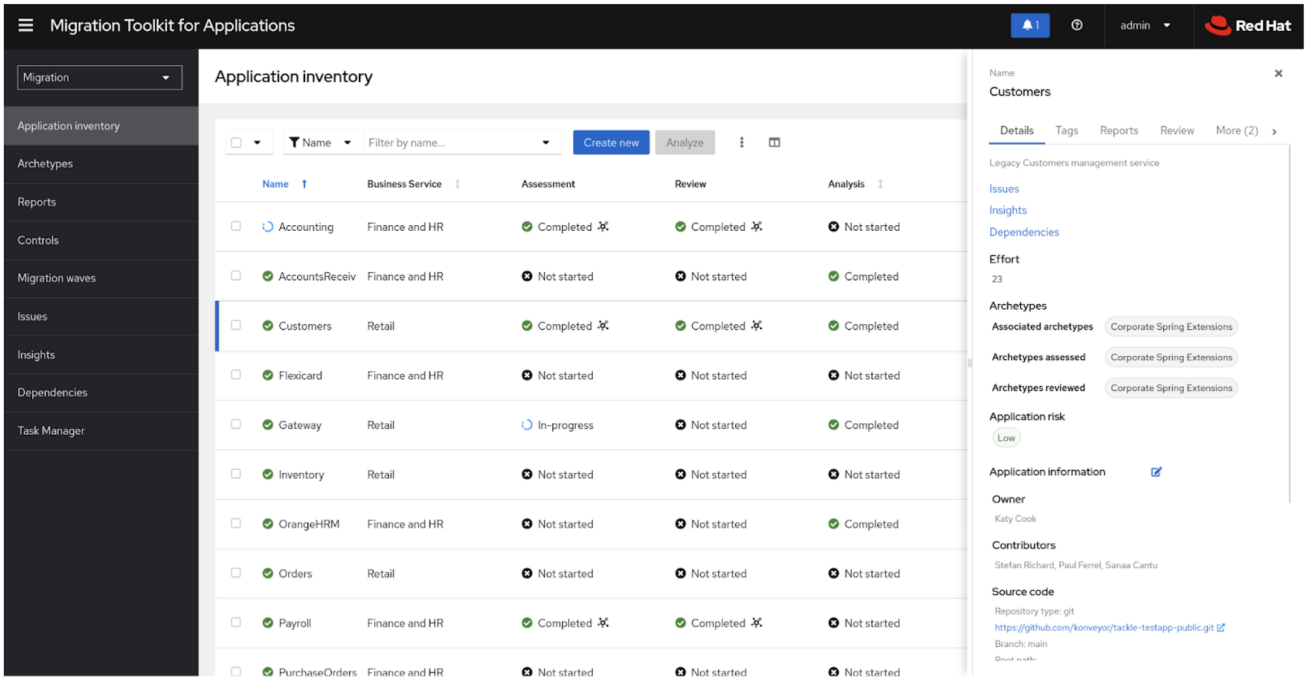


Figure 4. Application Inventory (Click the image to view it in full size in a new browser tab)

Applications can be categorized using:

- Business domains
- Technology stacks
- Tags and metadata
- Dependencies
- Business criticality

This information helps architects determine which applications should be modernized first and which migrations require additional planning.

Assessing Migration Readiness

Before development teams begin modernization, MTA evaluates each application and estimates the migration effort.

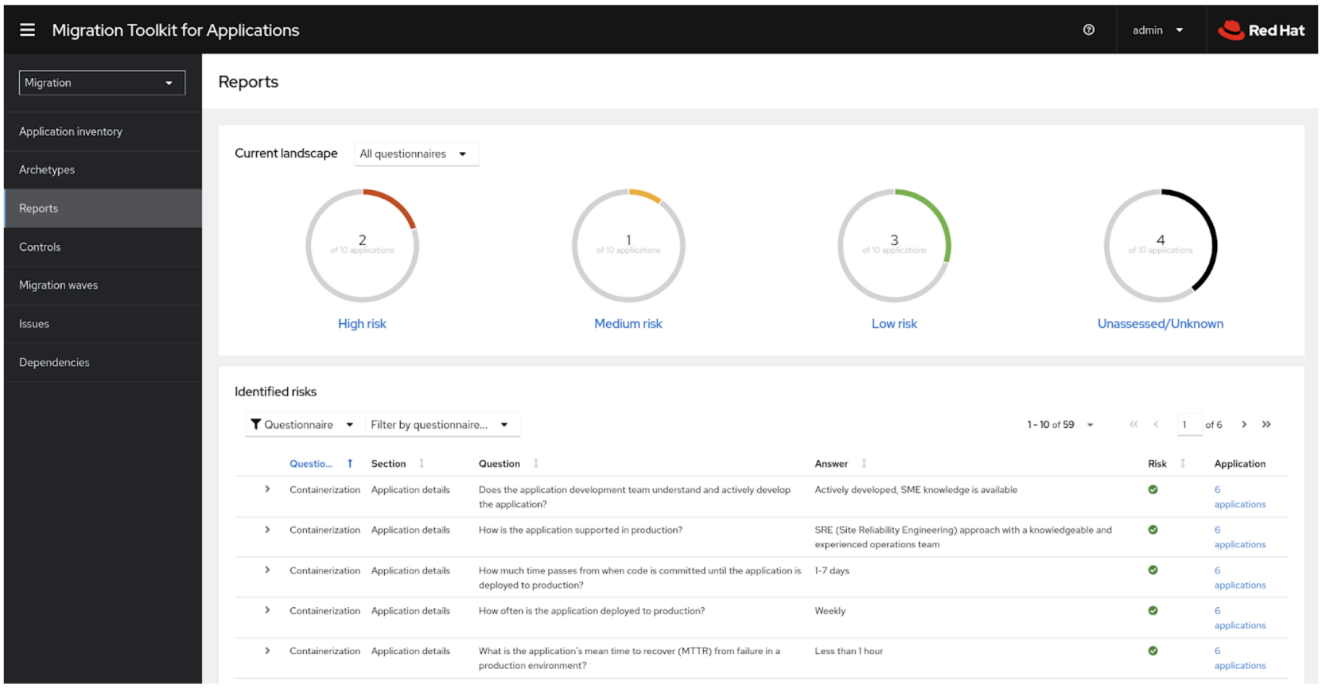


Figure 5. Application Assessment Report (Click the image to view it in full size in a new browser tab)

Assessment reports help organizations:

- Estimate modernization effort
- Identify migration risks
- Detect unsupported technologies
- Recommend the most appropriate modernization strategy

This allows teams to prioritize migrations based on both technical complexity and business value.

Automated Application Analysis

Once priorities have been established, MTA performs a detailed analysis of application source code.

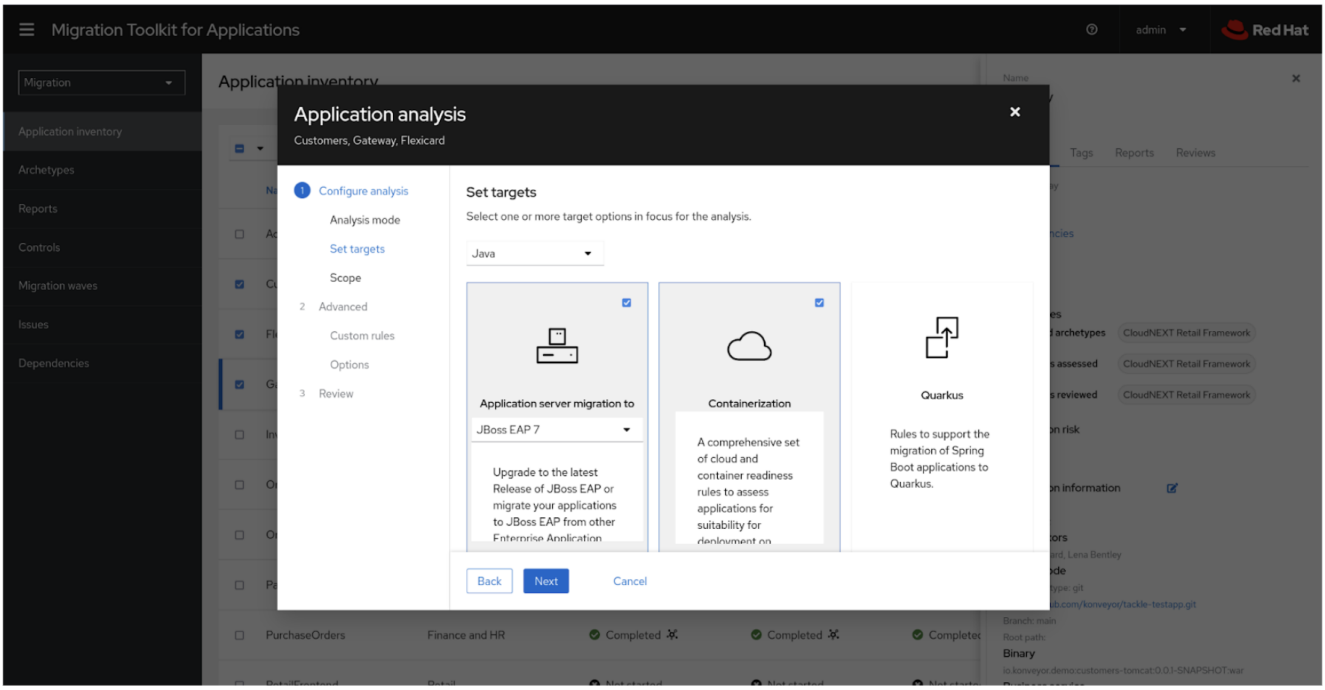


Figure 6. Application Analysis (Click the image to view it in full size in a new browser tab)

Using predefined migration rules, MTA automatically:

- Detects incompatible technologies
- Identifies required code changes
- Estimates migration effort
- Generates detailed migration reports

Instead of manually reviewing thousands of source files, developers receive actionable guidance that accelerates modernization.

Custom Migration Rules

Every organization has its own internal frameworks, libraries, coding standards, and architectural guidelines.

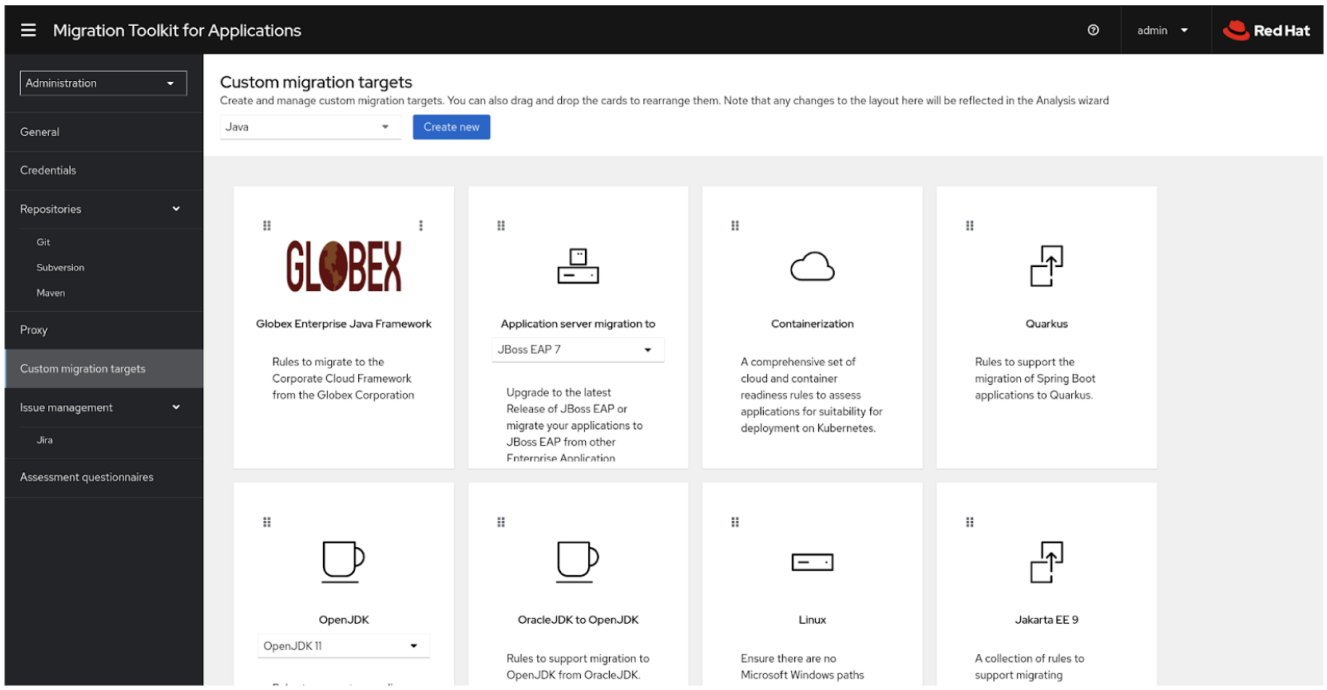


Figure 7. Custom Migration Rules (Click the image to view it in full size in a new browser tab)

MTA allows architects to extend the built-in rule engine with custom migration rules that can:

- Detect organization-specific technologies
- Recommend internal modernization patterns
- Link directly to internal documentation
- Reuse migration knowledge across projects

Over time, these rules become an internal migration "cookbook" that improves every future modernization effort.

Business Value

By standardizing modernization across the enterprise, organizations gain:

- Consistent migration processes across teams
- Portfolio-wide visibility into modernization efforts
- Better prioritization based on business value and complexity
- More accurate migration effort estimates
- Reusable migration knowledge and best practices
- Faster modernization through automation and shared guidance



Migration Toolkit for Applications is more than a code analysis tool.

It provides a repeatable modernization framework that helps organizations inventory, assess, prioritize, analyze, and modernize large application portfolios using consistent engineering practices.

Reflection

▼ *Think about your own organization (Click to expand)*



- Approximately how many applications does your organization maintain?
- Which applications would you modernize first, and why?
- How are migration priorities determined today?
- Could migration knowledge be shared across multiple teams?
- How would a standardized Migration Factory improve modernization efforts?

```
<div class="key-card">
```

Key Takeaway

Scaling application modernization requires more than analyzing individual applications. Migration Toolkit for Applications enables organizations to assess entire application portfolios, standardize migration practices, share reusable knowledge, and coordinate modernization across multiple teams.

```
</div>
```

Lab Scenario

Business Scenario

You are part of a platform engineering team responsible for modernizing the organization's business-critical **Coolstore** retail application.

The application currently runs as a containerized workload on **JBoss EAP 7** in OpenShift. While it continues to operate reliably, it depends on legacy **Java EE (`javax`)** APIs, making it incompatible with modern **Jakarta EE**-based runtimes.

To align with current enterprise standards and supported Red Hat runtimes, the organization has decided to migrate the application to **JBoss EAP 8**. The primary migration effort involves replacing legacy `javax` APIs with their `jakarta` equivalents while preserving existing business functionality.

Rather than manually reviewing thousands of source files, your team will use **Migration Toolkit for Applications (MTA)** to analyze the application and identify the required migration changes. You will then use **Red Hat Developer Lightspeed** to generate AI-assisted code recommendations that accelerate the modernization effort.

Throughout the remaining labs in this course, you'll follow a modernization workflow similar to that used in enterprise application migration projects—from assessment and analysis through AI-assisted code modernization and validation.



This learning path demonstrates a common enterprise modernization scenario by migrating an application from **JBoss EAP** 7 to **JBoss EAP 8**. Although JBoss EAP 8 is the migration target in these labs, the same MTA assessment and AI-assisted modernization workflow can also be applied when migrating applications to other supported runtimes, such as **Red Hat build of Quarkus** or **Red Hat build of Spring Boot**.

For more information about migrating to JBoss EAP 8, see the JBoss EAP 8 Migration Guide.

Lab 1: MTA Prerequisites

In this lab, you'll configure the Red Hat Model-as-a-Service (MaaS) platform and create an API key for AI-assisted application modernization. You'll use this API key in the upcoming Migration Toolkit for Applications (MTA) labs.

By the end of this lab, you will have:

- Accessed the Red Hat MaaS platform
- Subscribed to the required Granite model
- Created and saved an API key for MTA

Configure the Red Hat MaaS Platform

Migration Toolkit for Applications integrates with Red Hat Developer Lightspeed to provide AI-assisted migration recommendations. Before using MTA, configure the Red Hat Model-as-a-Service (MaaS) platform and create an API key.

1. Open the [Red Hat MaaS Portal] in your browser and sign in using your Red Hat account.

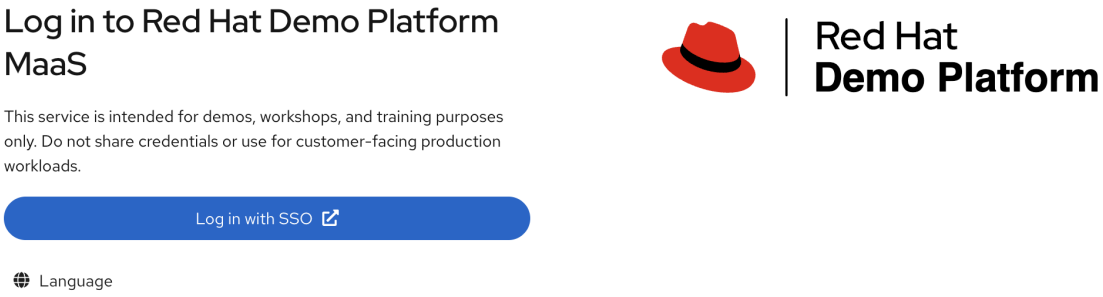


*Figure 8. Red Hat MaaS Portal(**Click the image to view it in full size in a new browser tab**)*

2. From the navigation menu, select **Models**.

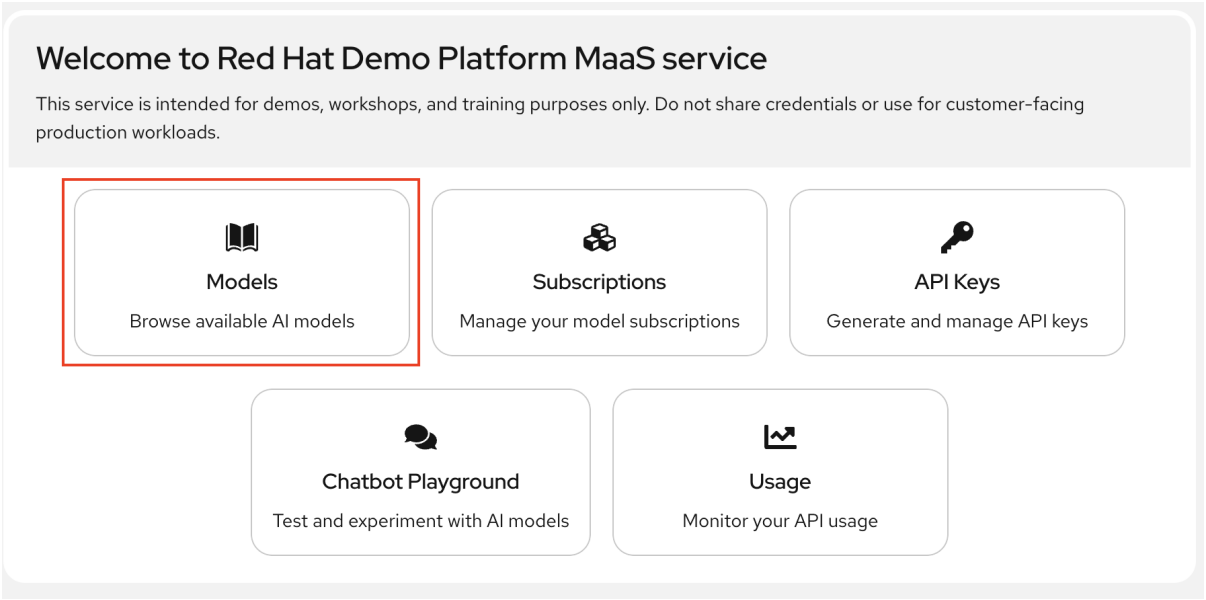


*Figure 9. Red Hat MaaS Portal Home page(**Click the image to view it in full size in a new browser tab**)*

3. Select **granite-3-2-8b-instruct** and click **[Subscribe]**.

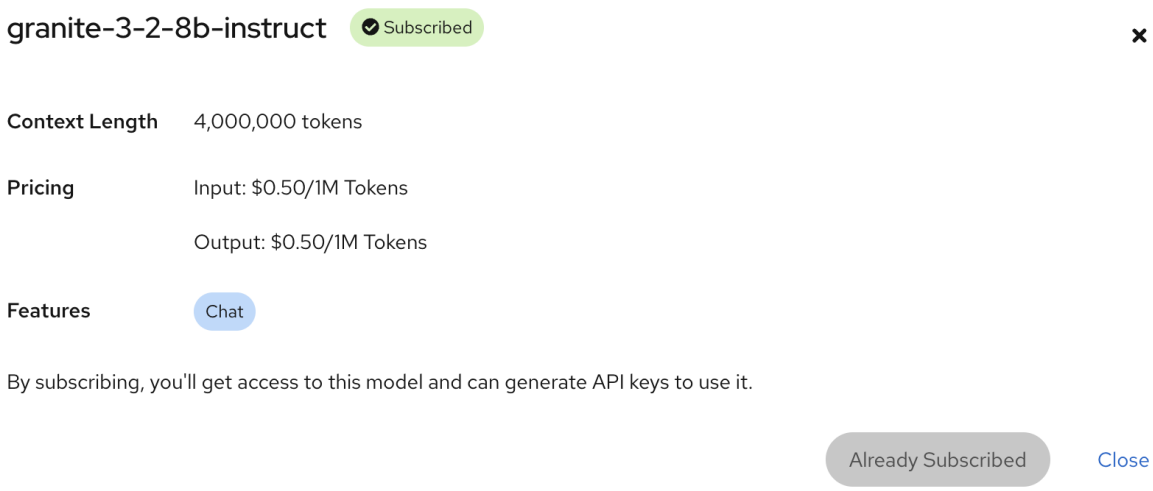


*Figure 10. Subscribe to Granite model(**Click the image to view it in full size in a new browser tab**)*

4. From the navigation menu, select **API Keys**.

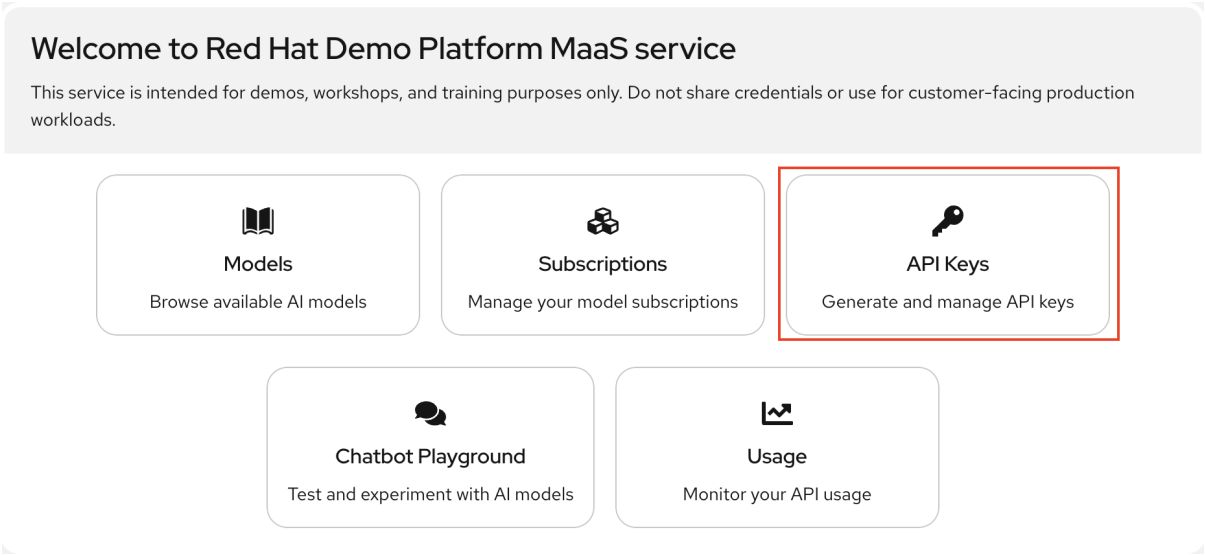


*Figure 11. Red Hat MaaS API Keys(**Click the image to view it in full size in a new browser tab**)*

5. Click **[Create API Key]**.

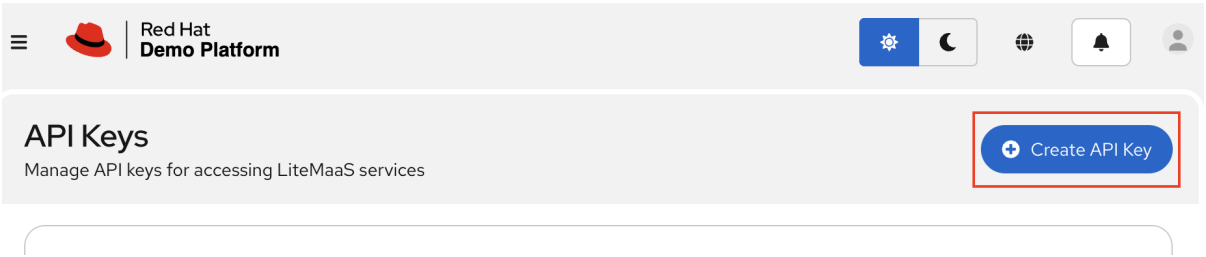


*Figure 12. Create API Key(**Click the image to view it in full size in a new browser tab**)*

6. Enter the following values and click **[Create API Key]**:
   - **Name**: `MTA-Dev`
   - **Description**: `AI-assisted modernization for MTA`
   - **Model**: `granite-3-2-8b-instruct`
7. Copy the generated API key and save it securely. You'll use this key in the upcoming MTA labs.

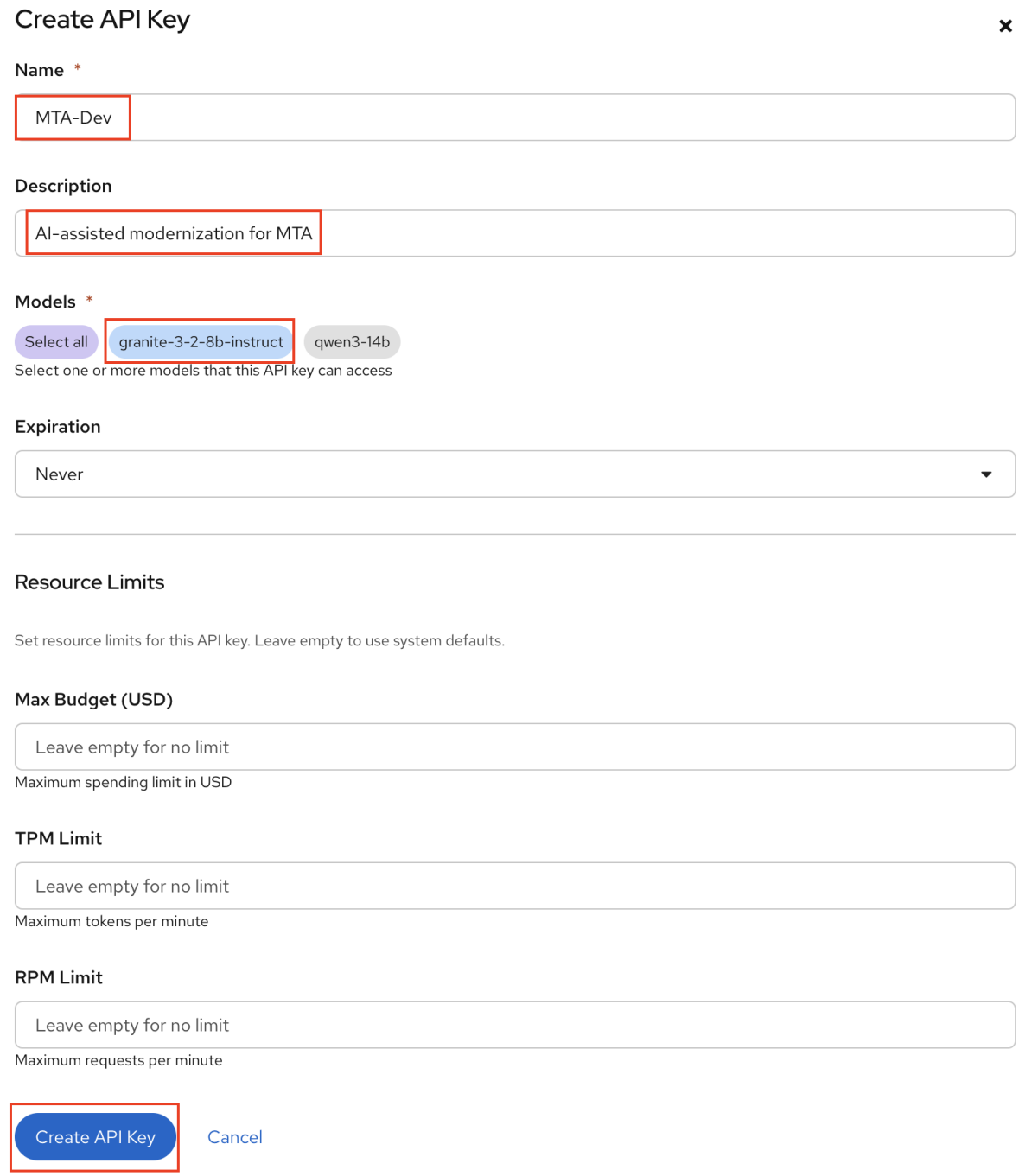


*Figure 13. Sample API Key Information(**Click the image to view it in full size in a new browser tab**)*

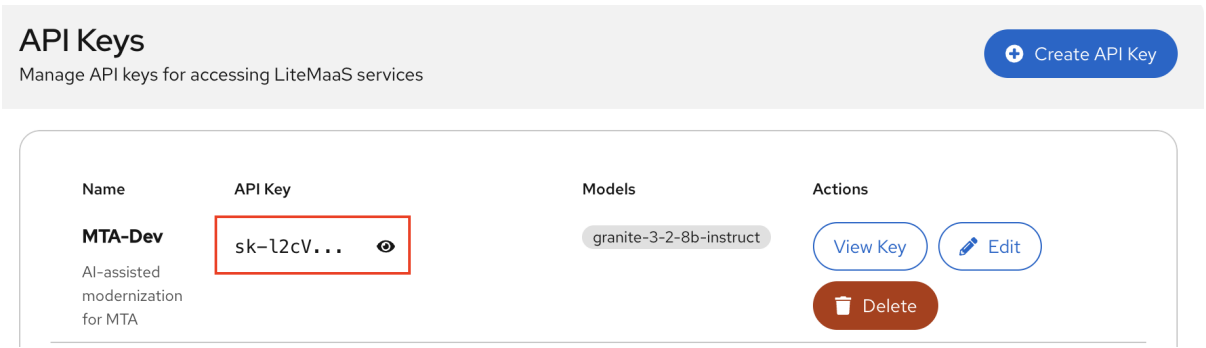


*Figure 14. Sample API Key(**Click the image to view it in full size in a new browser tab**)*

Lab 2: Install MTA

In this lab, you'll install the Migration Toolkit for Applications (MTA) Operator from OperatorHub and verify that the installation completes successfully.

By the end of this lab, you will have:

- Installed the Migration Toolkit for Applications Operator
- Verified that the MTA Operator is ready

Install the Migration Toolkit for Applications Operator

The Migration Toolkit for Applications Operator provides the components required to deploy and manage MTA on OpenShift.

1. Open the OpenShift web console.
2. Sign in using the `admin` credentials provided on the RHDP Lab Environment page.
3. From the navigation menu, select **[Ecosystem]** → **[Software Catalog]**.

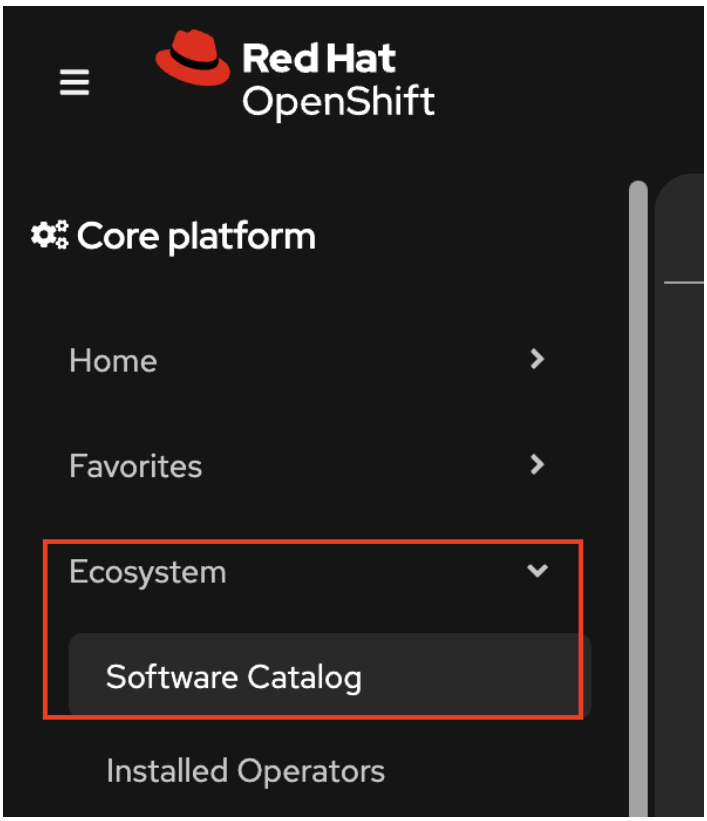


*Figure 15. Ecosystem → Software Catalog(***Click the image to view it in full size in a new browser tab***)*

4. Search for **Migration Toolkit for Applications** and select the operator. Then, click **[Install]**.

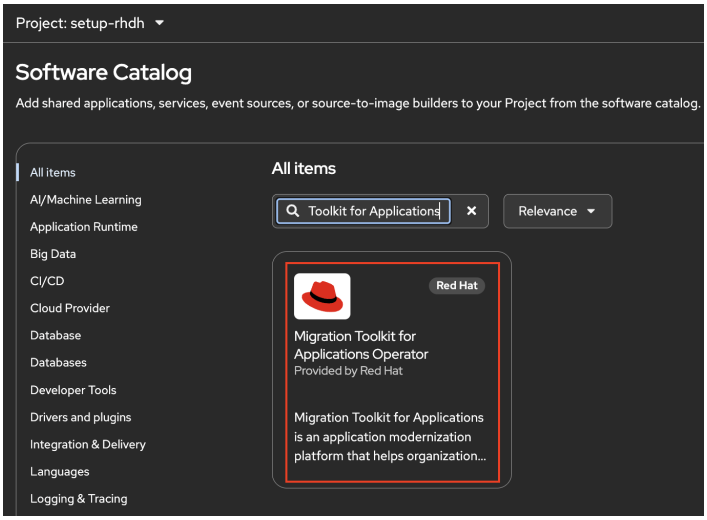


*Figure 16. Migration Toolkit for Applications Operator(**Click the image to view it in full size in a new browser tab**)*

5. Accept the default installation options and click **[Install]**.

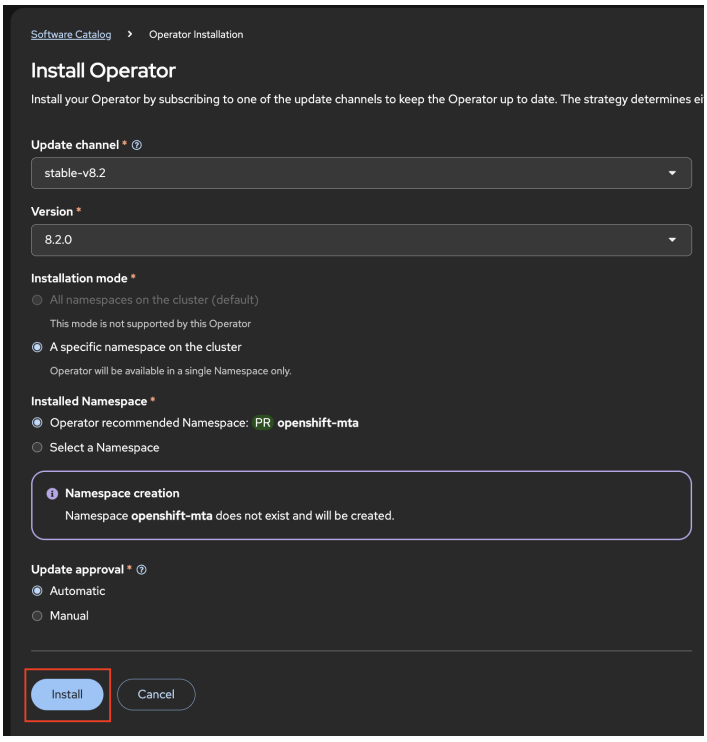


*Figure 17. Install Migration Toolkit for Applications Operator(**Click the image to view it in full size in a new browser tab**)*

6. Wait until the operator reaches the **Succeeded** state.

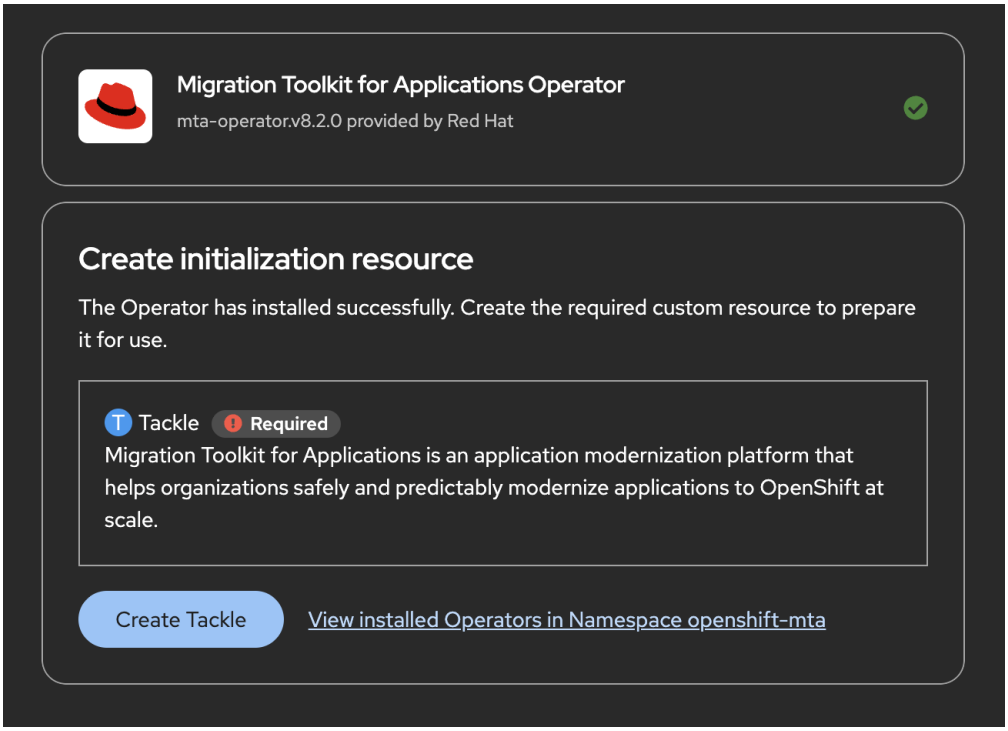


*Figure 18. Installing the MTA Operator(**Click the image to view it in full size in a new browser tab**)*



Do not create the `Tackle` custom resource during installation. You'll deploy it manually in the next exercise.

Verify the Installation

Verify that the MTA Operator is installed successfully.

```
oc get csv -n openshift-mta
```

Lab 3: Deploy MTA with Developer Lightspeed

In this lab, you'll deploy the Migration Toolkit for Applications (MTA) web console and configure it to use Developer Lightspeed for AI-assisted application modernization.

By the end of this lab, you will have:

- Configured the MaaS API key for MTA
- Deployed the MTA web console
- Verified that the MTA components are running
- Accessed the MTA web console

Deploy the MTA Web Console

The MTA web console is deployed by creating a Tackle custom resource. Before deploying it, create a Secret containing the **MaaS API** key created in the previous lab.

```
oc create secret generic kai-api-keys \
  -n openshift-mta \
  --from-literal=OPENAI_API_BASE='https://maas-rhdp
-frontend.apps.maas.redhatworkshops.io/v1' \
  --from-literal=OPENAI_API_KEY='<YOUR_MTA_API_KEY>'
```

Create the Tackle custom resource to deploy the MTA web console.

```
cat <<EOF | oc apply -f -
apiVersion: tackle.konveyor.io/v1alpha1
kind: Tackle
metadata:
  name: mta
  namespace: openshift-mta
spec:
  feature_auth_required: 'false'
  feature_isolate_namespace: "false"
  maven_data_volume_size: "10Gi"
  analyzer_container_limits_cpu: "1"
  analyzer_container_limits_memory: "4Gi"
  analyzer_container_requests_cpu: "500m"
  analyzer_container_requests_memory: "2Gi"
  hub_container_limits_cpu: "2"
  hub_container_limits_memory: "4Gi"
  hub_container_requests_cpu: "500m"
  hub_container_requests_memory: "2Gi"
  kai_llm_proxy_enabled: true
  kai_solution_server_enabled: true
  kai_llm_provider: openai
  kai_llm_model: granite-3-2-8b-instruct
  kai_llm_baseurl: "https://maas-rhdp.apps.maas.redhatworkshops.io/v1"
  kai_solution_server_container_limits_cpu: "1"
  kai_solution_server_container_limits_memory: "2Gi"
  kai_solution_server_container_requests_cpu: "500m"
  kai_solution_server_container_requests_memory: "1Gi"
  kai_llm_proxy_container_limits_cpu: "1"
  kai_llm_proxy_container_limits_memory: "2Gi"
  kai_llm_proxy_container_requests_cpu: "500m"
  kai_llm_proxy_container_requests_memory: "1Gi"
EOF
```

Wait until all MTA pods are running.

```
oc get pods -n openshift-mta -w
```

Verify that the following MTA components are running: * `kai-api` * `kai-db` * `llm-proxy` * `mta-hub` * `mta-ui`

Retrieve the MTA route and open the web console in your browser.

```
oc get route -n openshift-mta
```

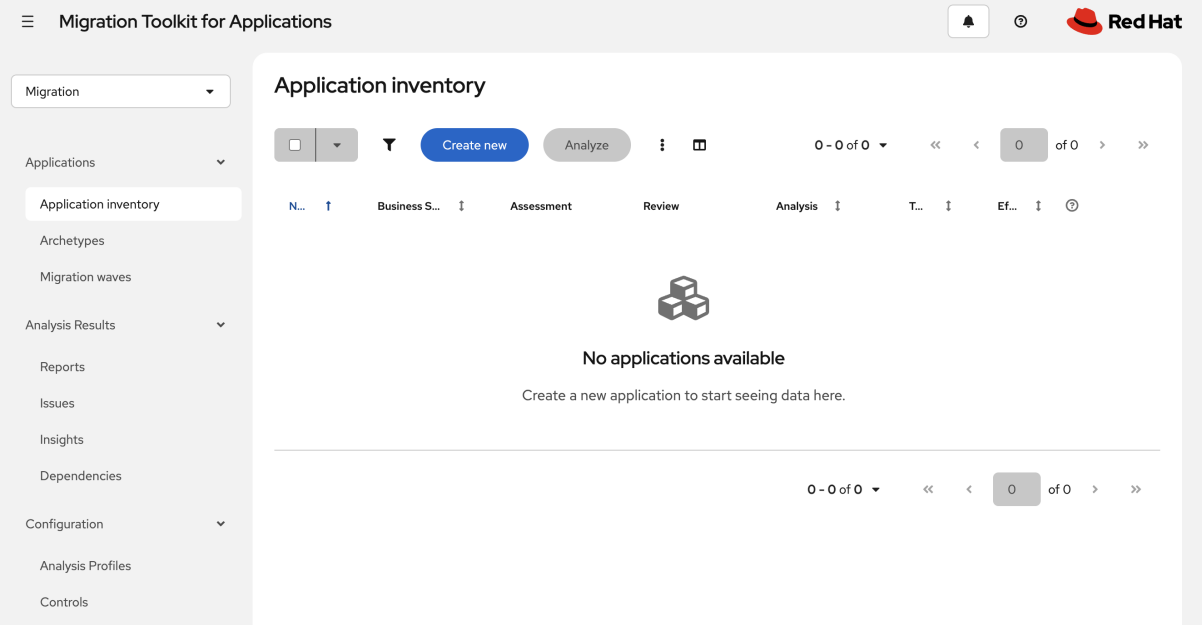


Figure 19. Migration Toolkit for Applications Web Console(Click the image to view it in full size in a new browser tab)

Lab 4: Configure GitLab Access

In this lab, you'll configure GitLab credentials in Migration Toolkit for Applications (MTA) so it can access the application source code for analysis and modernization.

By the end of this lab, you will have:

- Configured GitLab credentials in MTA
- Enabled MTA to access the application source code

Configure GitLab Access

Before importing the application, configure GitLab credentials so MTA can access the application source code.

1. In the MTA web console, navigate to **[Administration]** → **[Credentials]** → **[Create new]**.

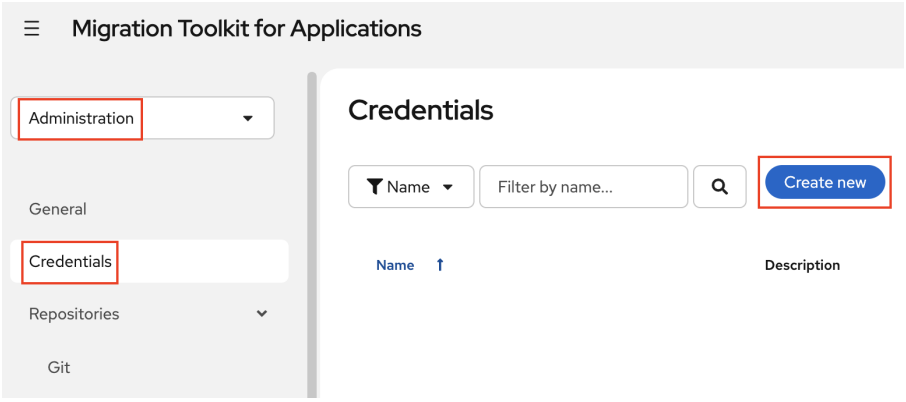


*Figure 20. Create New Credential(**Click the image to view it in full size in a new browser tab**)*

2. Enter the following values:
    - **Name**: `gitlab`
    - **Type**: `Source Control`
    - **Authentication**: `Username/Password`
    - **Default Credential**: Enabled
    - **Username**: `platform-admin`
    - **Password**: `openshift`
3. Click **[Create]**.

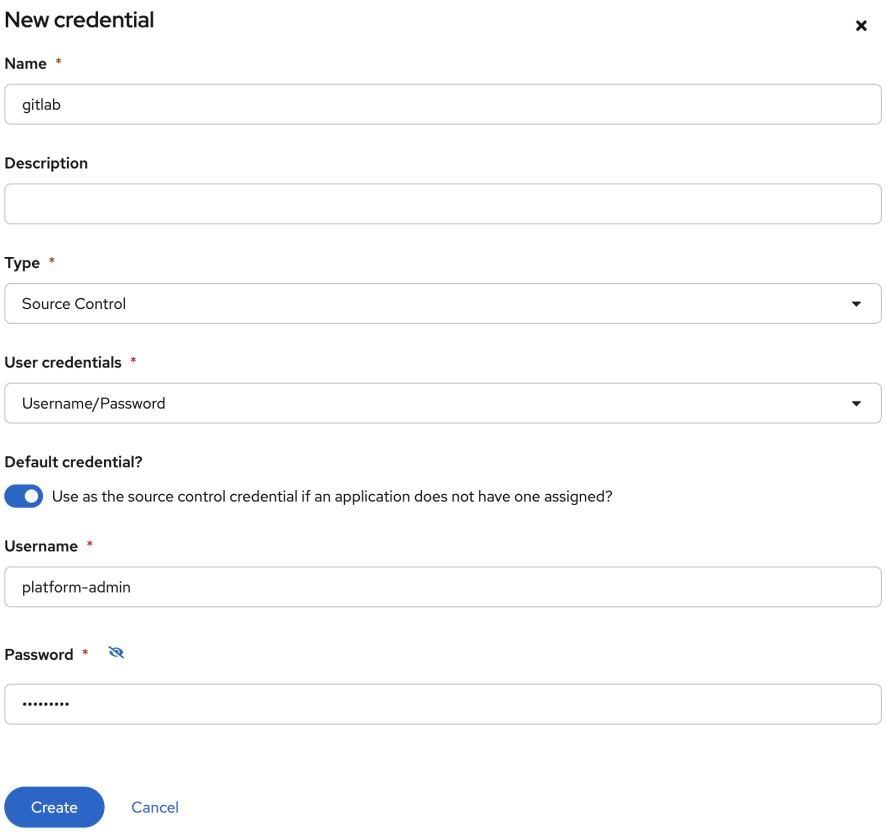


*Figure 21. GitLab Credential(**Click the image to view it in full size in a new browser tab**)*

Lab 5: Import the Coolstore Application

In this lab, you'll add the Coolstore application to the Migration Toolkit for Applications (MTA) Application Inventory and configure its GitLab source repository for analysis.

By the end of this lab, you will have:

- Created a Coolstore application entry in MTA
- Connected the application to its GitLab source repository
- Configured the application branch and runtime tag

Import the Coolstore Application

Create an application entry for the Coolstore application in the MTA Application Inventory.

1. In the MTA web console, navigate to **[Migration]** → **[Application Inventory]** → **[Create new]**.

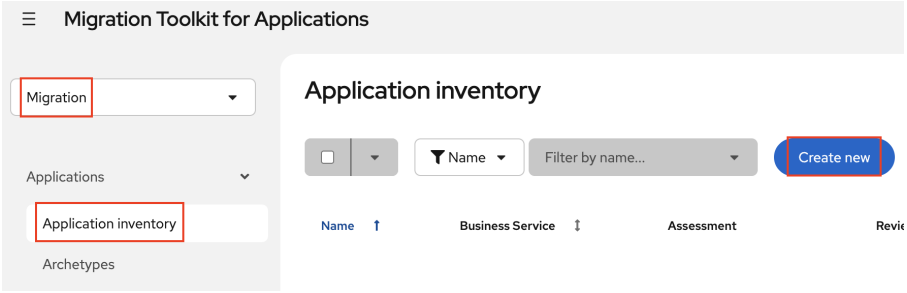


Figure 22. Application Inventory(Click the image to view it in full size in a new browser tab)

2. Enter the following values and click **[Create]**.
   - **Name**: `coolstore`
   - **Manual Tags**: `Runtime/EAP`
   - **Source Repository**: `{gitlab_url}/mta-apps/coolstore.git`
   - **Branch**: `main`

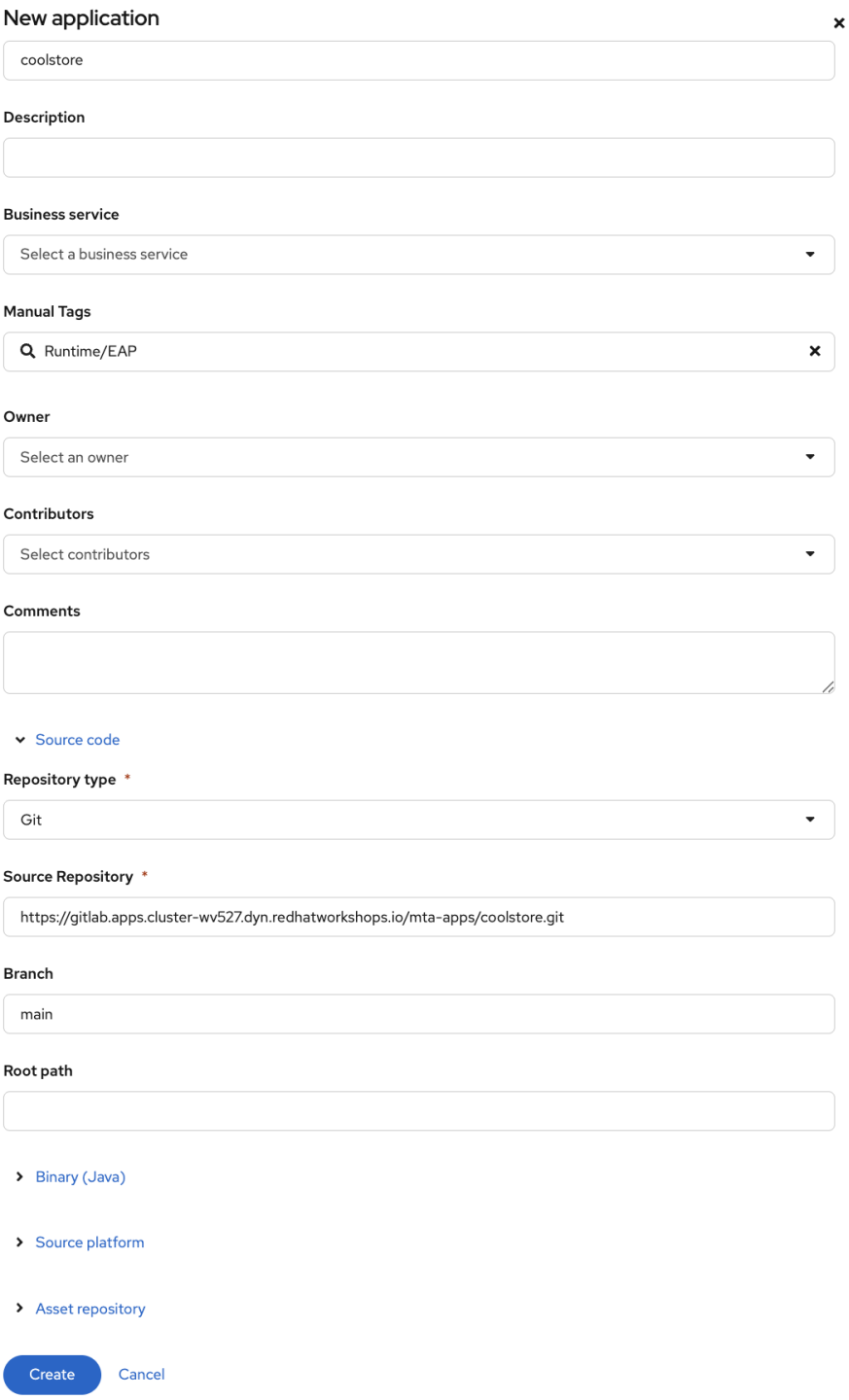


Figure 23. Create Application(Click the image to view it in full size in a new browser tab)

Lab 6: Analyze the Coolstore Application

In this lab, you'll analyze the Coolstore application with Migration Toolkit for Applications (MTA), targeting JBoss EAP 8. You'll review the migration findings and identify the changes required to modernize the application.

By the end of this lab, you will have:

- Analyzed the Coolstore application for JBoss EAP 8
- Reviewed the generated migration report

- Identified issues requiring migration changes
- Understood the javax to jakarta migration requirements

Analyze the Application

Run an application analysis targeting JBoss EAP 8.

1. Select the coolstore application.
2. Click **[Analyze]**.

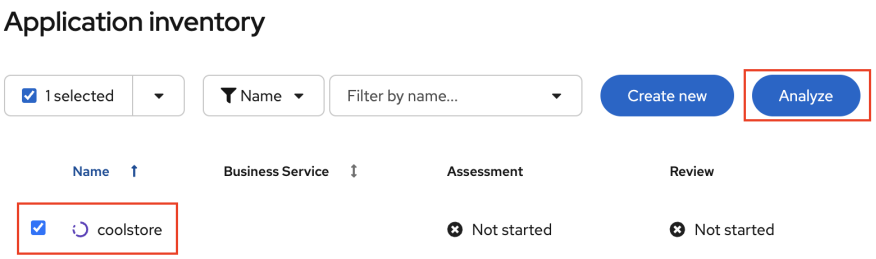


Figure 24. Start Analysis(Click the image to view it in full size in a new browser tab)

3. Configure the analysis using the following options:

   Mode: Manual Selection Target Runtime: JBoss EAP 8 Scope: Application and internal dependencies only

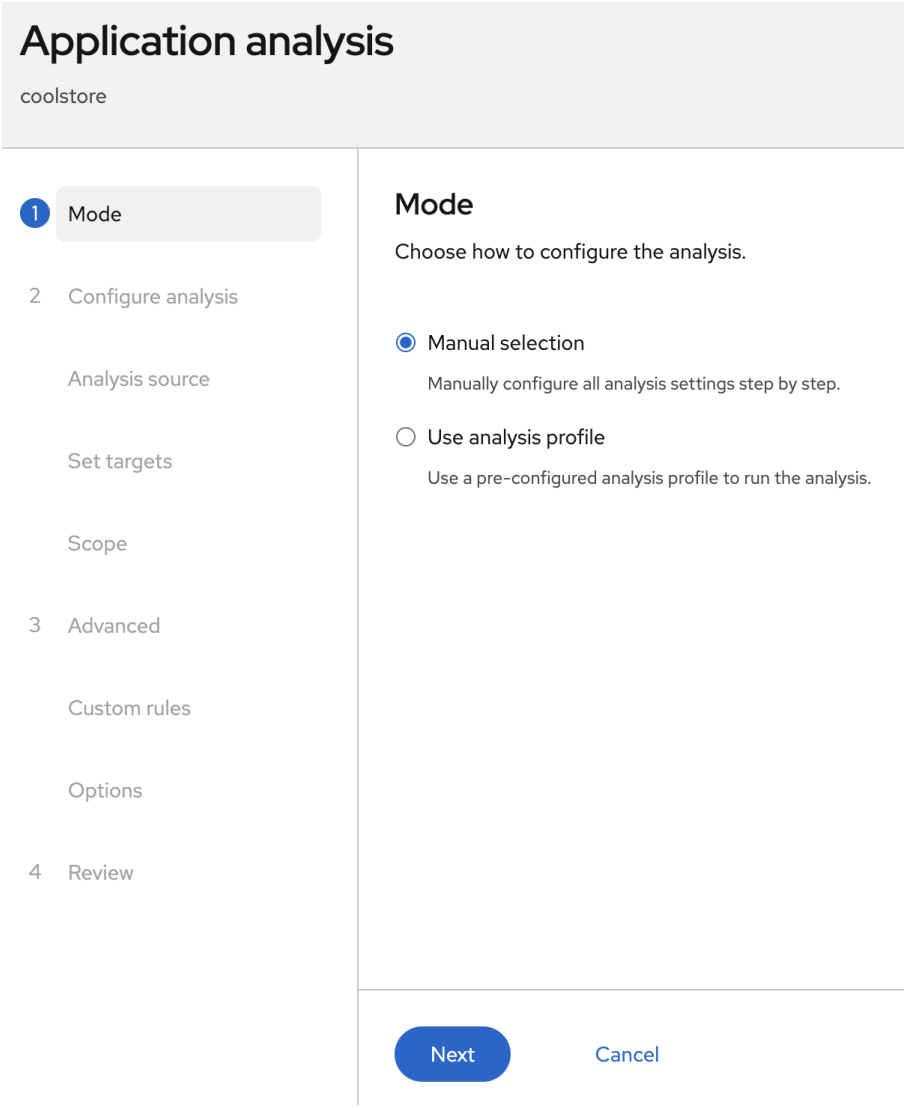


Figure 25. Mode settings(Click the image to view it in full size in a new browser tab)

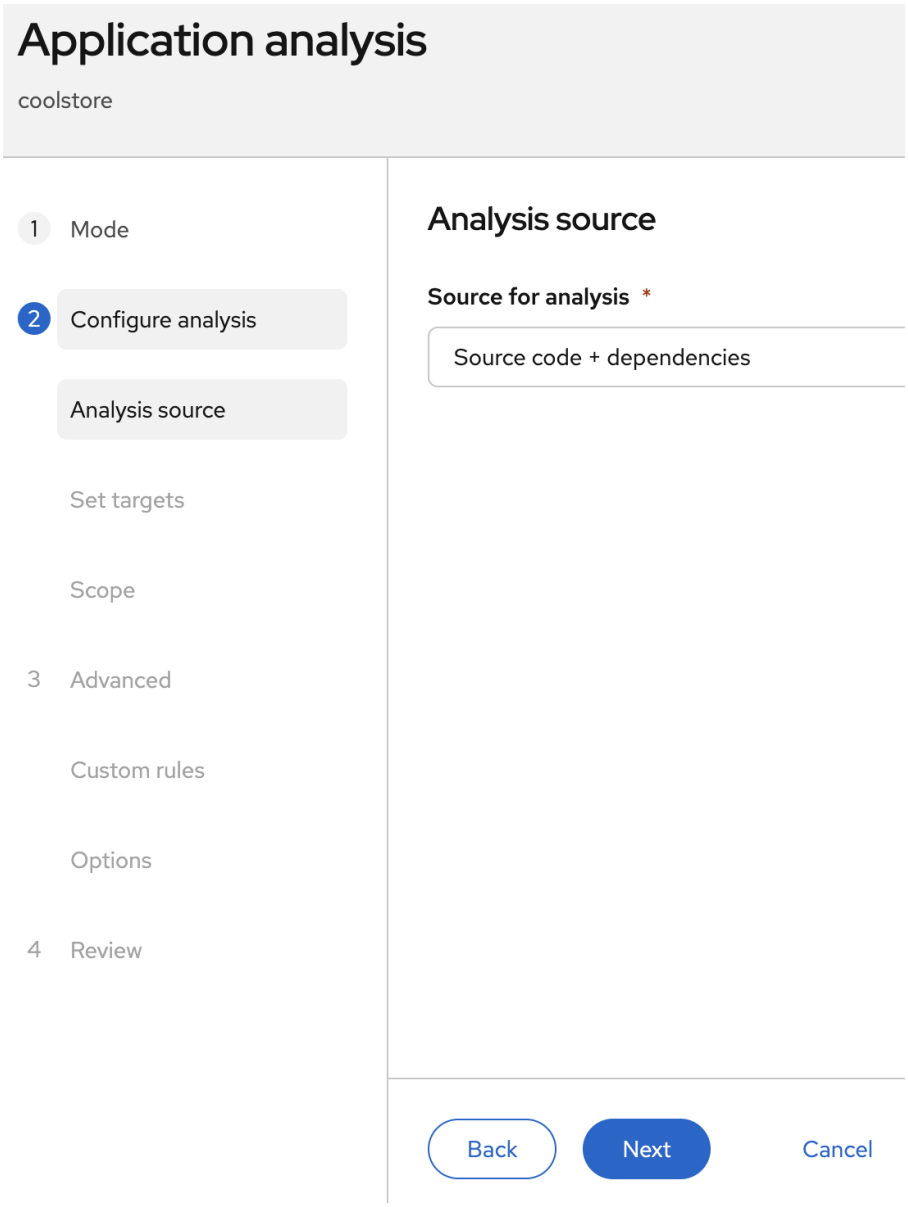


Figure 26. Analysis settings(Click the image to view it in full size in a new browser tab)

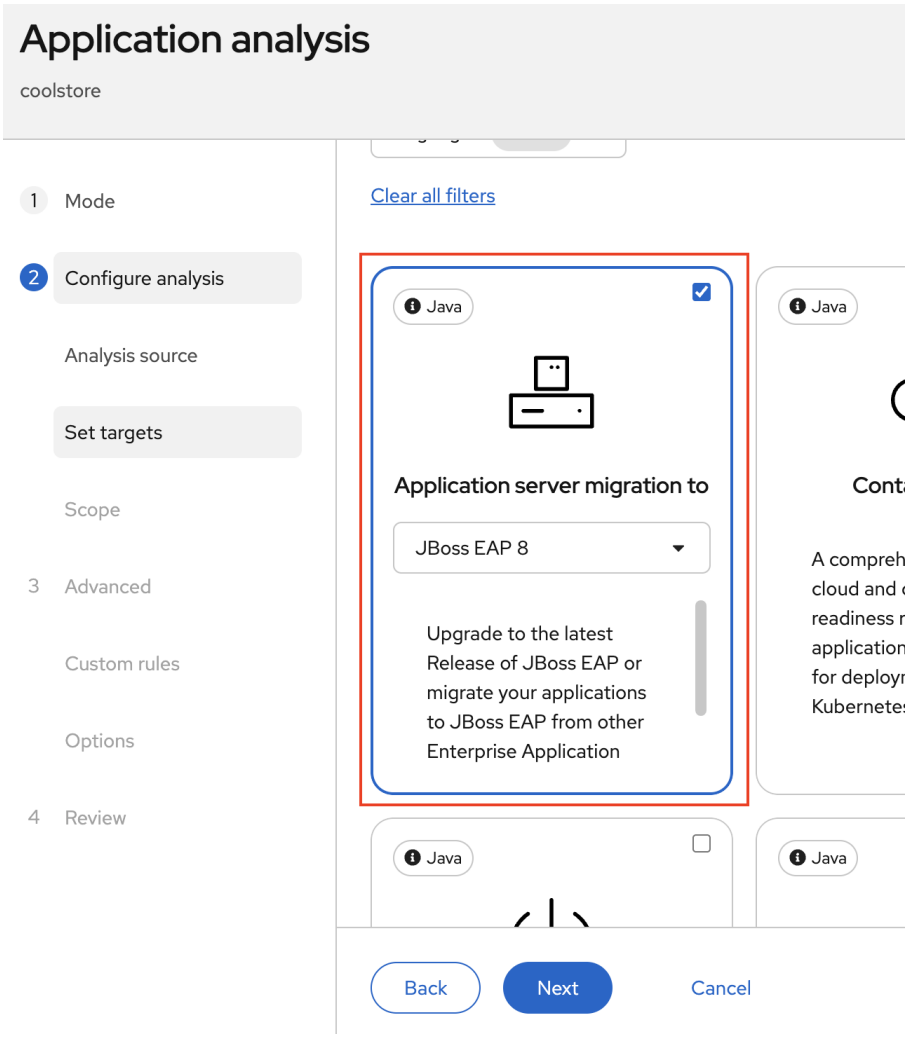


Figure 27. Set targets settings(Click the image to view it in full size in a new browser tab)

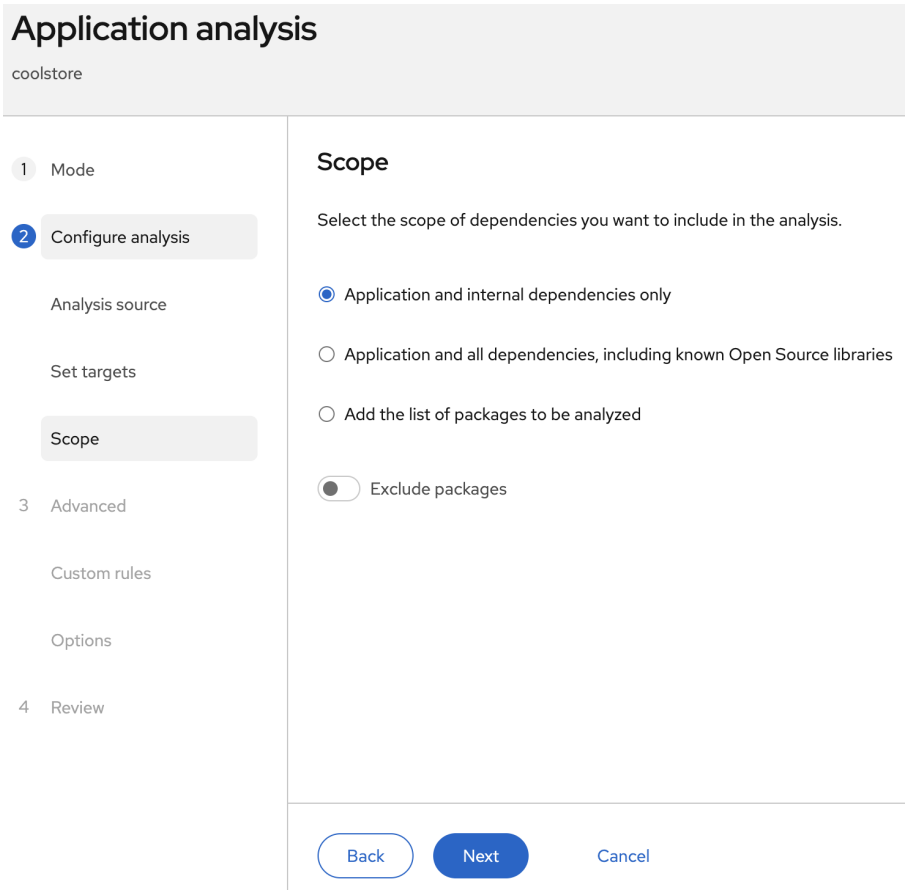


Figure 28. Scope settings(Click the image to view it in full size in a new browser tab)

4. Accept the remaining defaults and click **[Run]**.

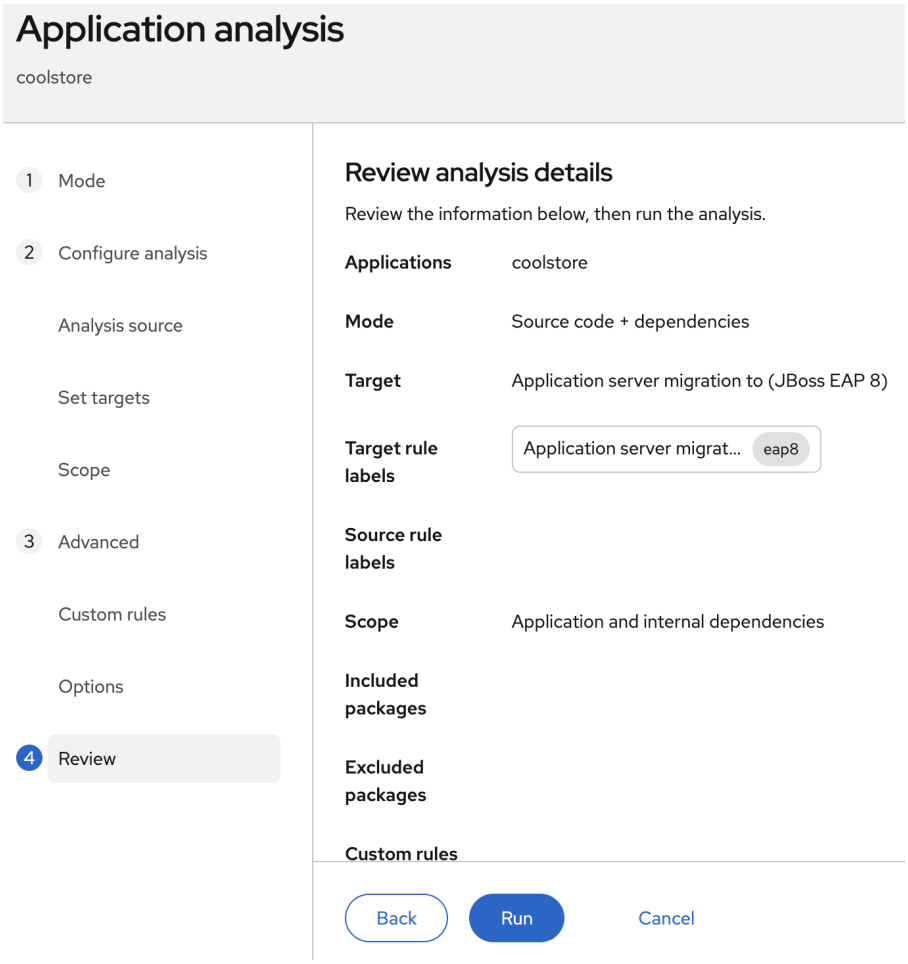


Figure 29. Review Analysis details(Click the image to view it in full size in a new browser tab)

The analysis scans the application's source code, dependencies, and configuration files using the built-in MTA migration rules.

1. Wait for the analysis to complete.

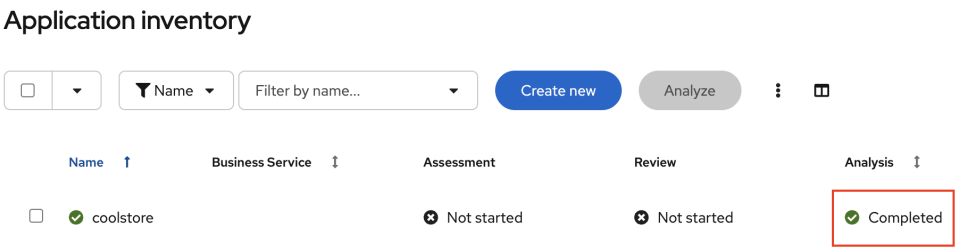


Figure 30. Analysis Complete(Click the image to view it in full size in a new browser tab)

Review the Analysis Report

Once the analysis completes, review the migration findings.

1. Open the coolstore application.
2. Select the Issues tab.

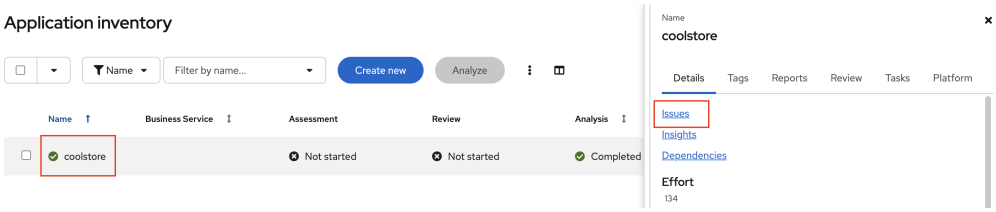


Figure 31. Analysis Report(Click the image to view it in full size in a new browser tab)

3. Review the reported issues:
    - Mandatory — Changes required for the migration to succeed.
    - Potential — Recommended improvements.
    - Information — Additional guidance and documentation.

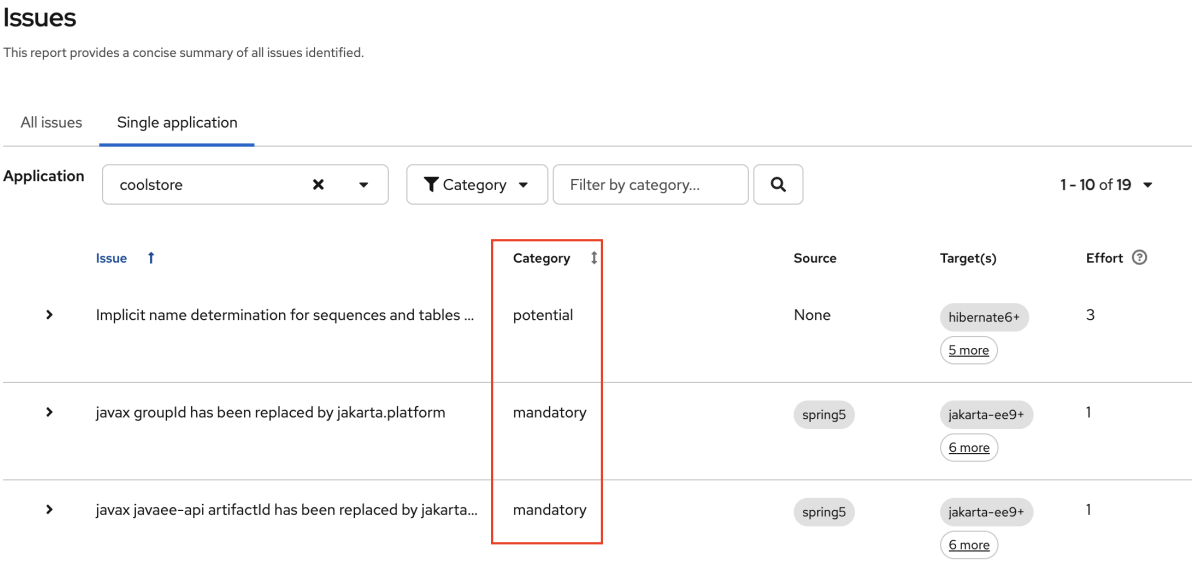


Figure 32. Migration Issue Categories(Click the image to view it in full size in a new browser tab)



The migration report identifies code changes required to migrate the application from Java EE (javax) to Jakarta EE (jakarta). In the next lab, you'll use Red Hat Developer Lightspeed to generate AI-assisted recommendations for addressing these issues.

Lab Summary

In this section, you prepared and used **Migration Toolkit for Applications (MTA)** to assess the Coolstore application for migration from **JBoss EAP** 7 to **JBoss EAP 8**.

You started by configuring the **Red Hat Model-as-a-Service (MaaS)** platform and creating an API key to enable AI-assisted application modernization. You then installed the **MTA Operator** and deployed the MTA web console with Developer Lightspeed integration.

Next, you configured **GitLab access** in MTA and added the Coolstore application to the MTA Application Inventory. You performed an application analysis targeting **JBoss EAP 8** and reviewed the generated migration report to identify compatibility issues and required migration changes.

By completing this section, you have:

- Configured Red Hat MaaS for AI-assisted modernization
- Installed and deployed Migration Toolkit for Applications
- Configured GitLab access for application source code
- Added the Coolstore application to the MTA Application Inventory
- Analyzed the application for migration to JBoss EAP 8
- Reviewed migration findings and identified required changes
- Understood the **Java EE (`javax`) to Jakarta EE (`jakarta`)** migration requirements

The next section builds on this foundation by using **Red Hat OpenShift Dev Spaces** and the **MTA extension** to perform AI-assisted analysis and apply migration recommendations directly in the development environment.

MTA with Red Hat Openshift Dev Spaces



What's in this section?

In this section, you'll use Red Hat OpenShift Dev Spaces as the development environment for modernizing the Coolstore application with Migration Toolkit for Applications (MTA).

You'll install and configure the MTA extension in Dev Spaces, connect it to the MTA web console, create a migration profile for JBoss EAP 7 to JBoss EAP 8, and use Red Hat Developer Lightspeed to analyze the application and apply AI-assisted migration recommendations directly in the IDE.

Developer Lightspeed for MTA

The Challenge

In the previous lesson, you learned how Migration Toolkit for Applications (MTA) analyzes applications, identifies migration issues, and provides recommendations for modernization.

While MTA explains **what** needs to change, developers still need to manually implement many of those changes.

For large enterprise applications, this often involves updating hundreds—or even thousands—of similar code patterns.

Common questions developers face include:

- How should this migration issue be fixed?
- What is the recommended replacement for this deprecated API?
- Can repetitive code changes be automated?
- How can modernization remain consistent across multiple teams?

Applying migration recommendations manually can significantly increase modernization effort and project timelines.



Migration analysis identifies the work that needs to be done. AI helps developers complete that work faster by generating contextual code recommendations—but developers remain responsible for reviewing and validating every suggested change.

AI-Assisted Application Modernization

Red Hat Developer Lightspeed extends Migration Toolkit for Applications by using Artificial Intelligence to generate migration code suggestions based on MTA's analysis.

Instead of manually interpreting migration reports and implementing every recommendation, developers receive AI-assisted code suggestions directly within their development environment.

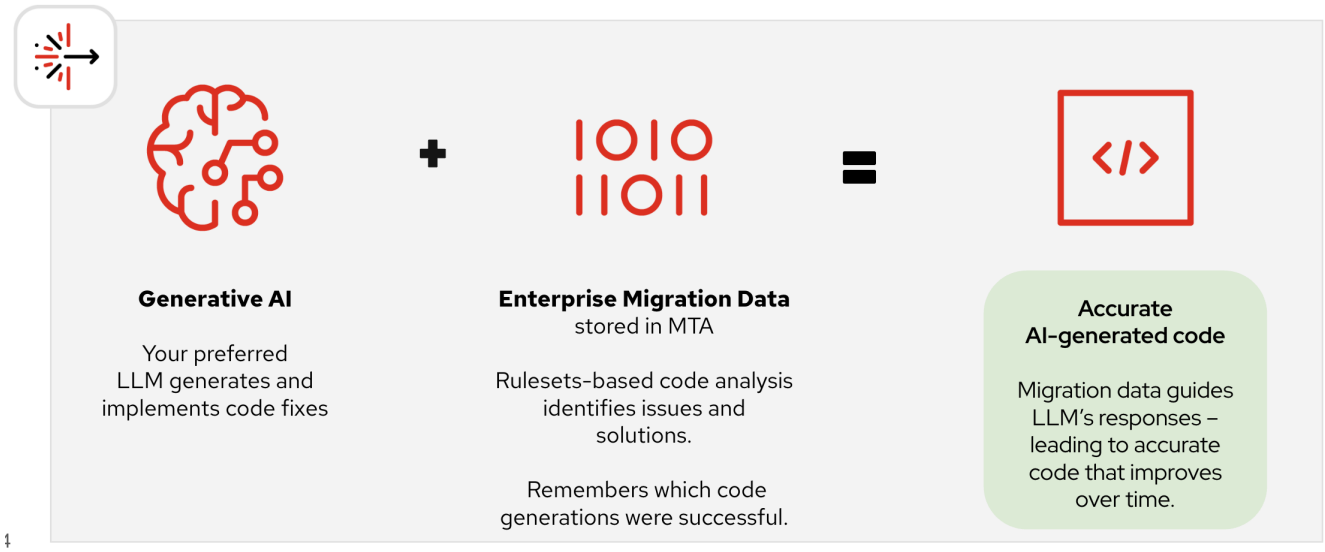


Figure 33. AI-Assisted Application Modernization (Click the image to view it in full size in a new browser tab)

By combining migration intelligence with generative AI, developers can modernize applications more quickly while maintaining control over the final implementation.

How Developer Lightspeed Works

Developer Lightspeed combines multiple sources of information to generate migration recommendations that are specific to your application.

Rather than acting as a generic coding assistant, it uses both enterprise migration knowledge and AI reasoning to produce context-aware suggestions.

Input	Purpose
Large Language Model (LLM)	Provides AI-powered reasoning and code generation capabilities.
MTA Migration Analysis	Supplies migration issues, rule violations, and recommended modernization targets.
Migration Rules	Provides enterprise migration guidance for supported runtimes and frameworks.
Application Context	Uses the application's existing source code and dependencies to generate relevant code suggestions.

Because recommendations are based on both AI and migration analysis, the generated code is more accurate and aligned with modernization goals.

AI-Powered Modernization Workflow

Modernizing an application with MTA and Developer Lightspeed follows a simple workflow.

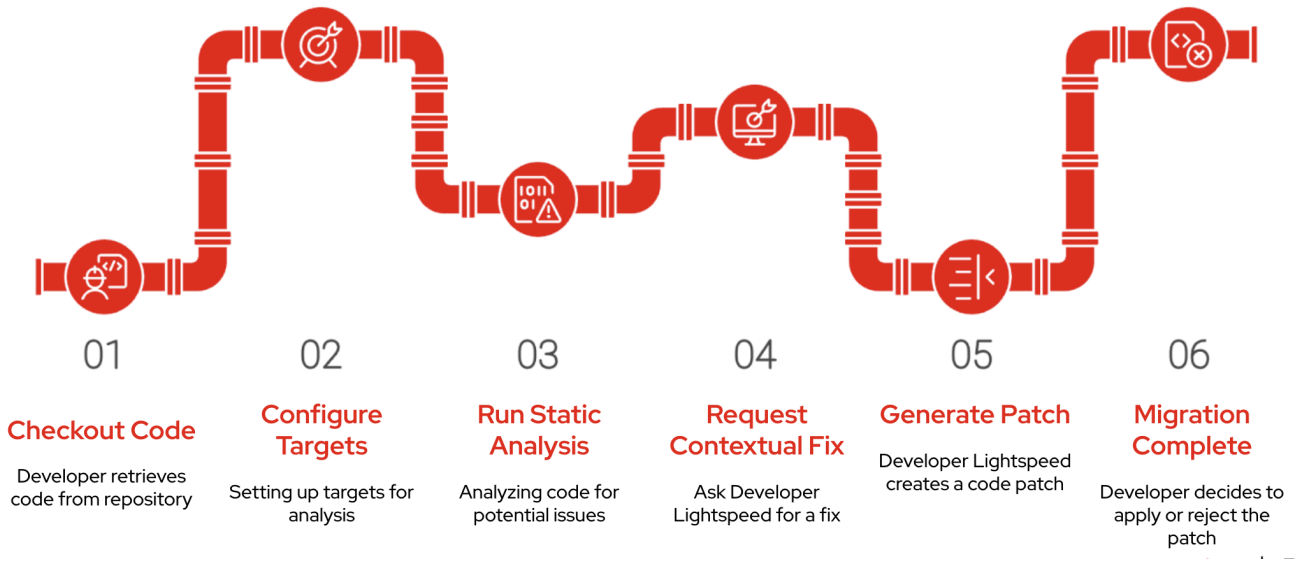


Figure 34. AI-Powered Modernization Workflow (Click the image to view it in full size in a new browser tab)

The workflow consists of the following stages.

Stage	Description
Checkout Code	Open the existing application source code from the Git repository.
Configure Migration Target	Select the desired modernization target, such as Quarkus or Jakarta EE.
Run Static Analysis	Migration Toolkit for Applications analyzes the application and identifies migration issues.
Request Contextual Fix	Developer Lightspeed generates AI-assisted recommendations for selected migration issues.
Generate Patch	The AI produces a suggested code patch using MTA findings and application context.
Review and Commit	Developers review, modify if necessary, and commit the accepted changes.

This workflow allows developers to combine automated migration analysis with AI-assisted implementation while retaining full control over the resulting code.

Business Value

Using Developer Lightspeed together with Migration Toolkit for Applications enables organizations to:

- Reduce repetitive manual migration work
- Generate code recommendations tailored to each application

- Apply migration fixes directly from the IDE
- Improve consistency across modernization projects
- Reuse migration knowledge across similar applications
- Increase developer productivity while maintaining human oversight



Developer Lightspeed does not automatically modify application code.

It assists developers by generating context-aware migration suggestions, while developers remain responsible for reviewing, validating, and accepting the proposed changes.

Why AI Matters

Enterprise modernization often involves thousands of repetitive code changes spread across multiple applications.

By combining AI with MTA's migration intelligence, organizations can:

- Accelerate modernization projects
- Reduce migration effort
- Improve consistency across teams
- Help developers focus on implementing business functionality instead of repetitive migration tasks

Rather than replacing developers, AI acts as an intelligent modernization assistant that helps teams deliver modernization projects more efficiently.

Reflection

▼ *Think about your own organization (Click to expand)*



- Which modernization tasks are repetitive within your organization?
- Could AI-generated migration suggestions reduce manual effort?
- Where would human review still be essential before accepting AI-generated code?
- How could AI help accelerate large-scale modernization projects?

```
<div class="key-card">
```

Key Takeaway

Migration Toolkit for Applications identifies **what** needs to change, while Red Hat Developer Lightspeed helps developers implement those changes by generating AI-assisted, context-aware code recommendations. Together, they accelerate application modernization while keeping

developers in control of the final code.

Lab 7: Install the MTA Extension in Dev Spaces

In this lab, you'll install and configure the Migration Toolkit for Applications (MTA) extension in your Dev Spaces workspace. You'll prepare the Coolstore application for migration analysis from JBoss EAP 7 to JBoss EAP 8 and enable AI-assisted remediation directly in the IDE.

By the end of this lab, you will have:

- Created a Dev Spaces workspace for the Coolstore application
- Installed the Language Support for Java extension
- Installed the MTA extension
- Connected the MTA extension to the MTA web console
- Created a migration profile for JBoss EAP 7 to JBoss EAP 8

Install the MTA Extension in Dev Spaces

The Coolstore application needs to be migrated from JBoss EAP 7 to JBoss EAP 8. You'll start by preparing your Dev Spaces workspace and installing the MTA extension to bring migration analysis and AI-assisted remediation directly into the IDE.

Access your Dev Spaces workspace

1. Retrive the GITLAB_URL as you shall be needing it in upcoming step.

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "$GITLAB_URL"
```

2. Retrieve the Openshift Dev Spaces dashboard URL and navigate to it using your browser.

```
oc get checluster devspaces \
  -n openshift-devspaces \
  -o jsonpath='{.status.cheURL}{"\n"}'
```

   Log in with your OpenShift credentials if prompted

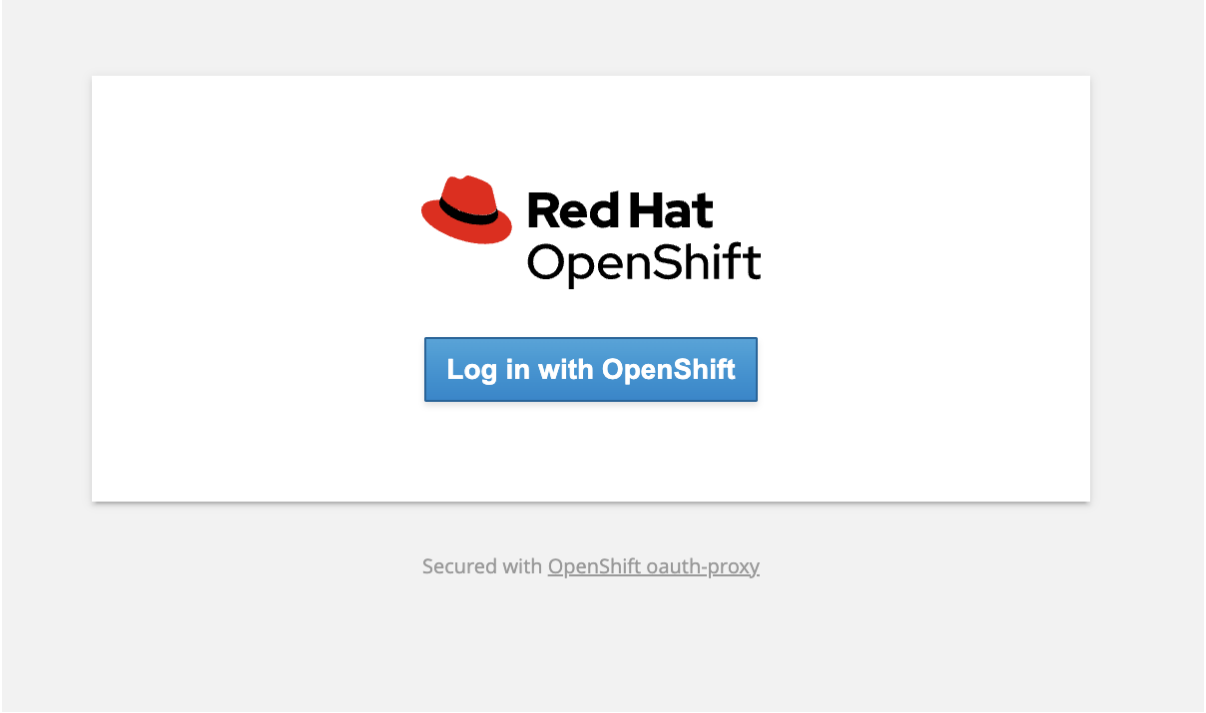


Figure 35. Red Hat Openshift Login Page(Click the image to view it in full size in a new browser tab)

Create a Dev Spaces workspace using the Coolstore application repository.

1. Open Dev Spaces and sign in using your OpenShift credentials if prompted.

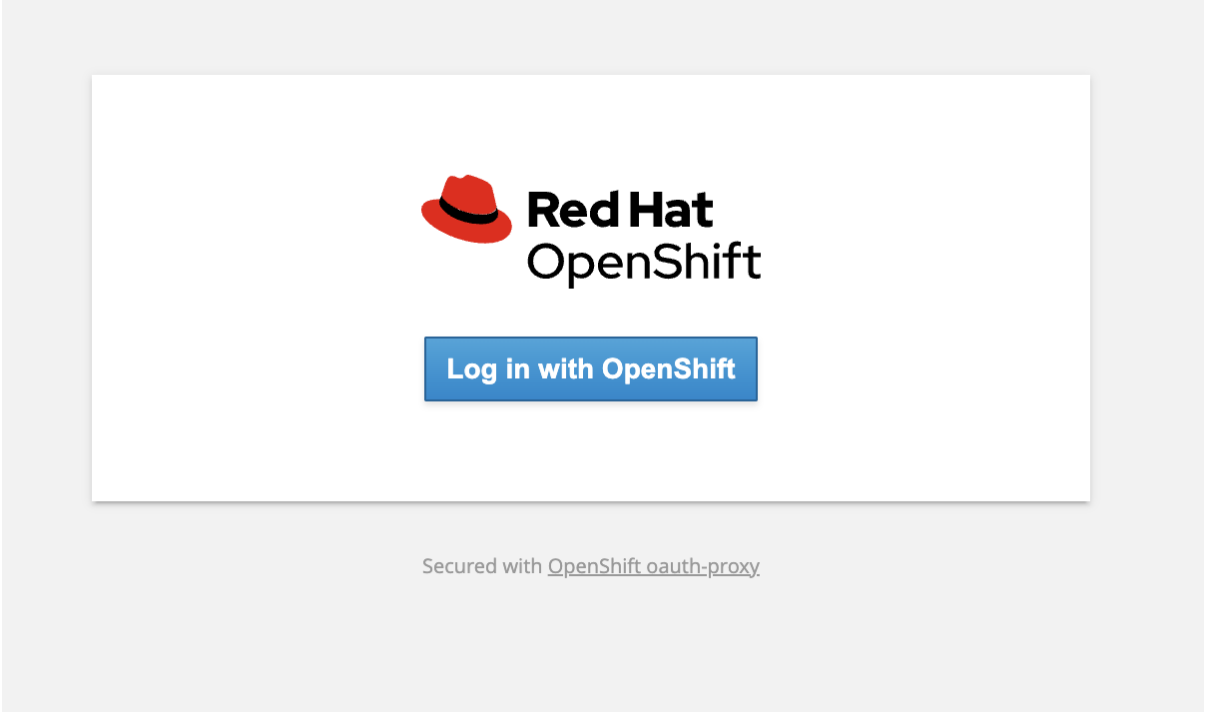


Figure 36. Red Hat OpenShift Login Page(Click the image to view it in full size in a new browser tab)

2. Click **[Add workspace]** to create a new workspace.

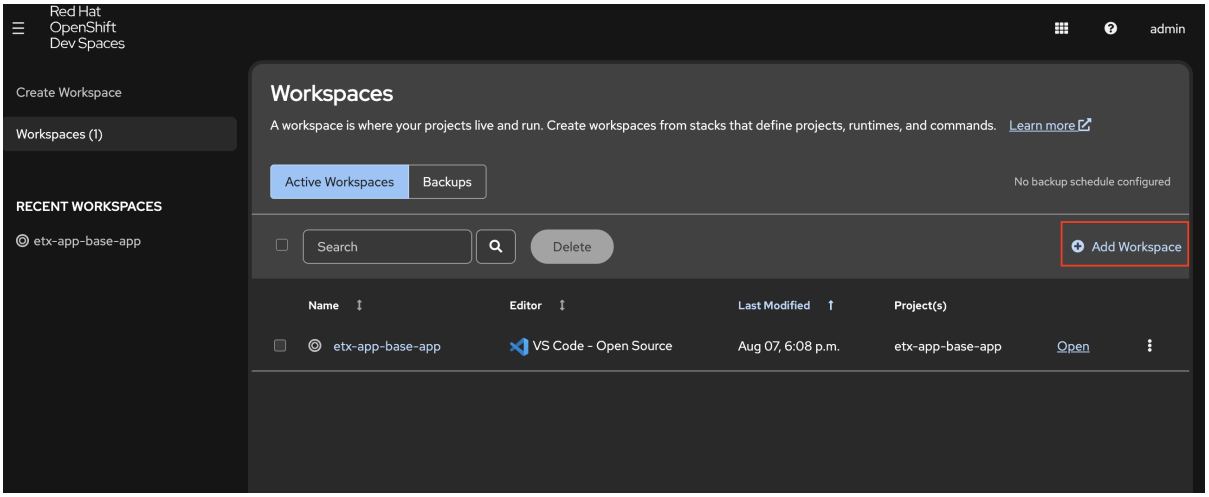


Figure 37. Create New Workspace(Click the image to view it in full size in a new browser tab)

3. Enter the following GitLab repository URL and click **[Create and Open]**:

   ${GITLAB_URL}/mta-apps/coolstore.git

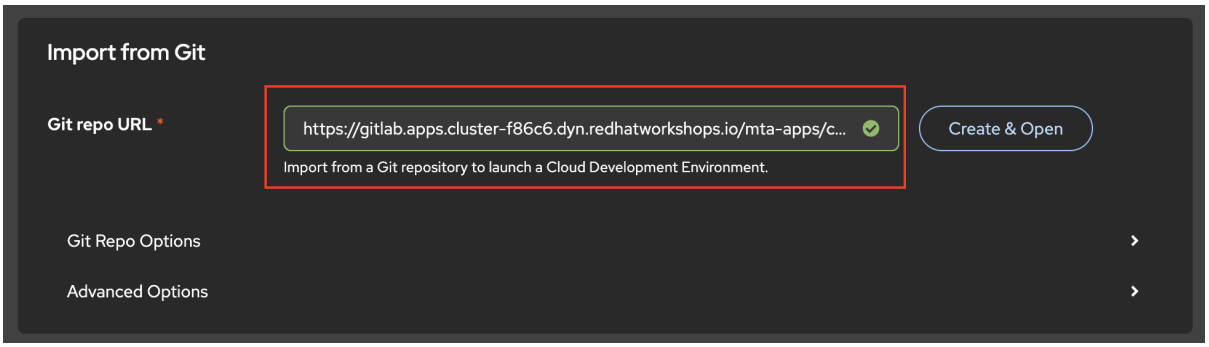


Figure 38. GitLab Repository for Coolstore(Click the image to view it in full size in a new browser tab)

4. Wait until the workspace is fully started and ready.

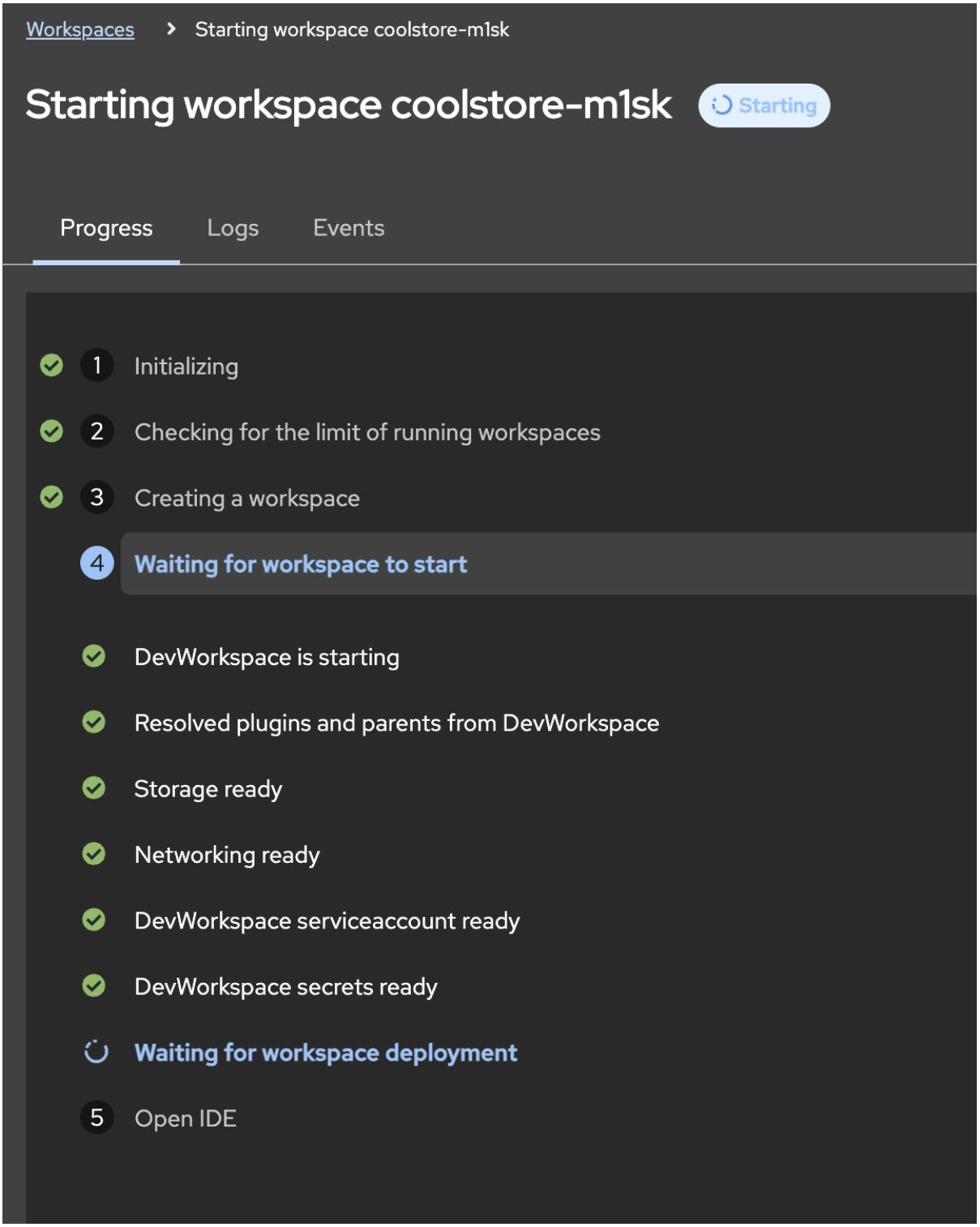


Figure 39. Workspace Creation Process(Click the image to view it in full size in a new browser tab)

Install the Java language support extension required to work with the Coolstore application.

1. In your Dev Spaces workspace, from the lower-left corner, select the **[Settings (gear) icon]**, and then select Extensions to open the Extensions Marketplace.

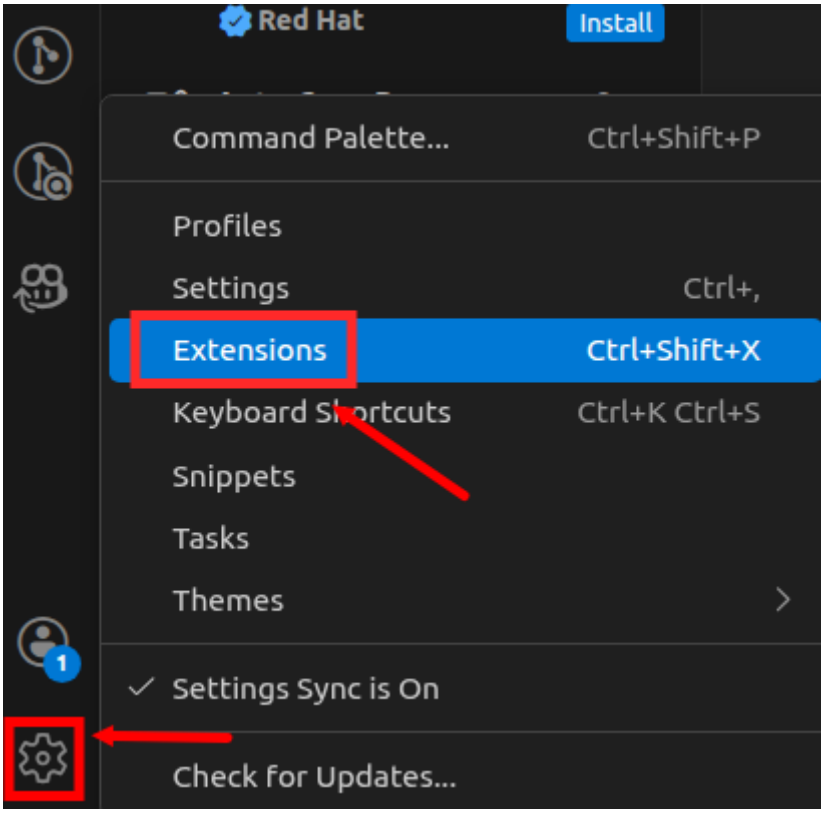


Figure 40. Extensions in VS Code(Click the image to view it in full size in a new browser tab)

2. In the Extensions search bar, search for Language Support for Java™ and install the extension.

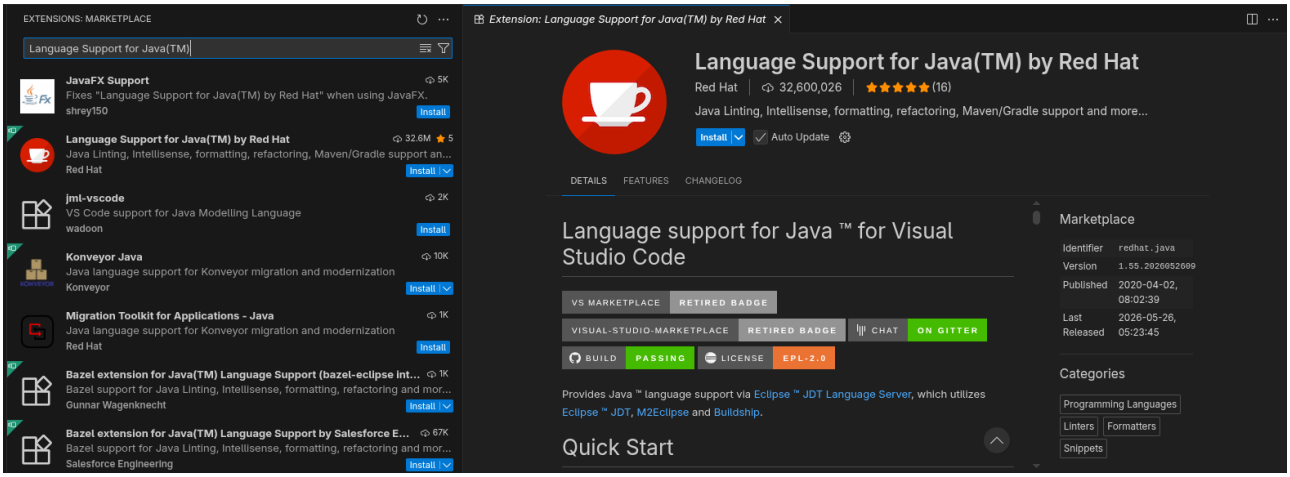


Figure 41. Language Support for Java Extension(Click the image to view it in full size in a new browser tab)

3. From the Status Bar, click the **[Java Lightweight Mode]** icon and switch the Java Language Server to Standard Mode.

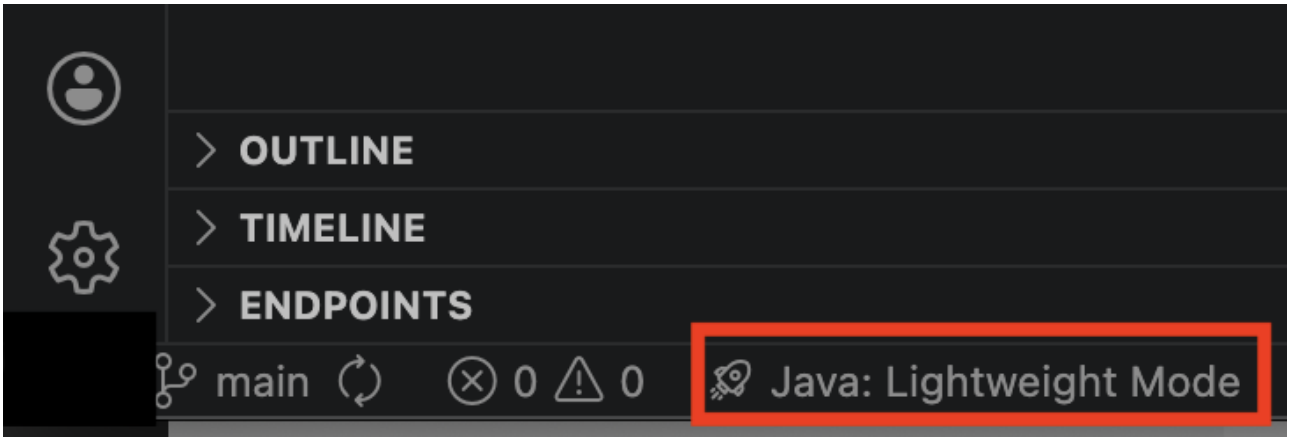


Figure 42. Switch to Standard Mode(Click the image to view it in full size in a new browser tab)

4. Wait for the code to compile and verify that the status shows Java: Ready.

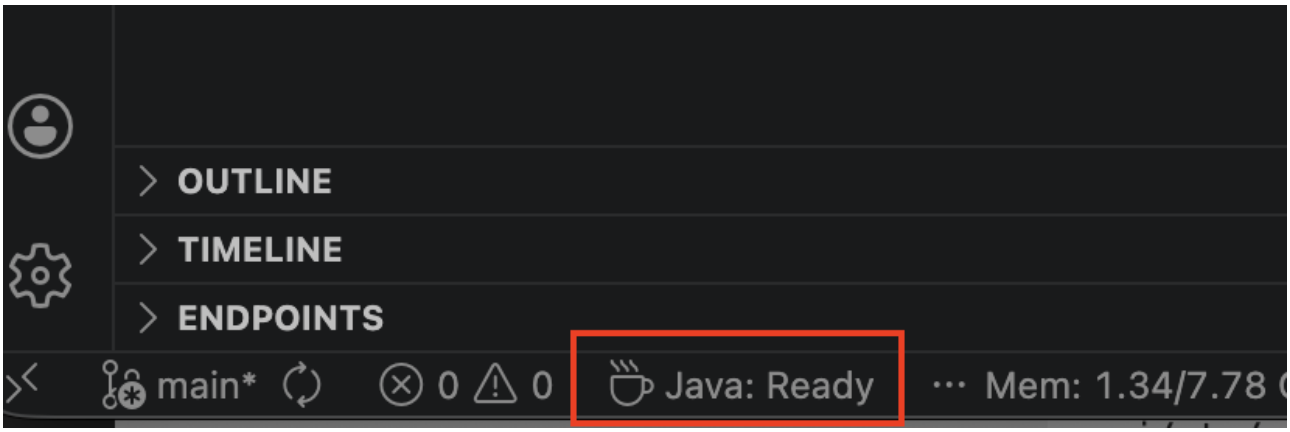


Figure 43. Sample Java: Ready State(Click the image to view it in full size in a new browser tab)

Install the MTA Extension



You may see two Migration Toolkit for Applications extensions. Select the extension shown in the screenshot with the new MTA logo.

1. In the Extensions search bar, search for **Migration Toolkit for Applications - Java** and install the extension.

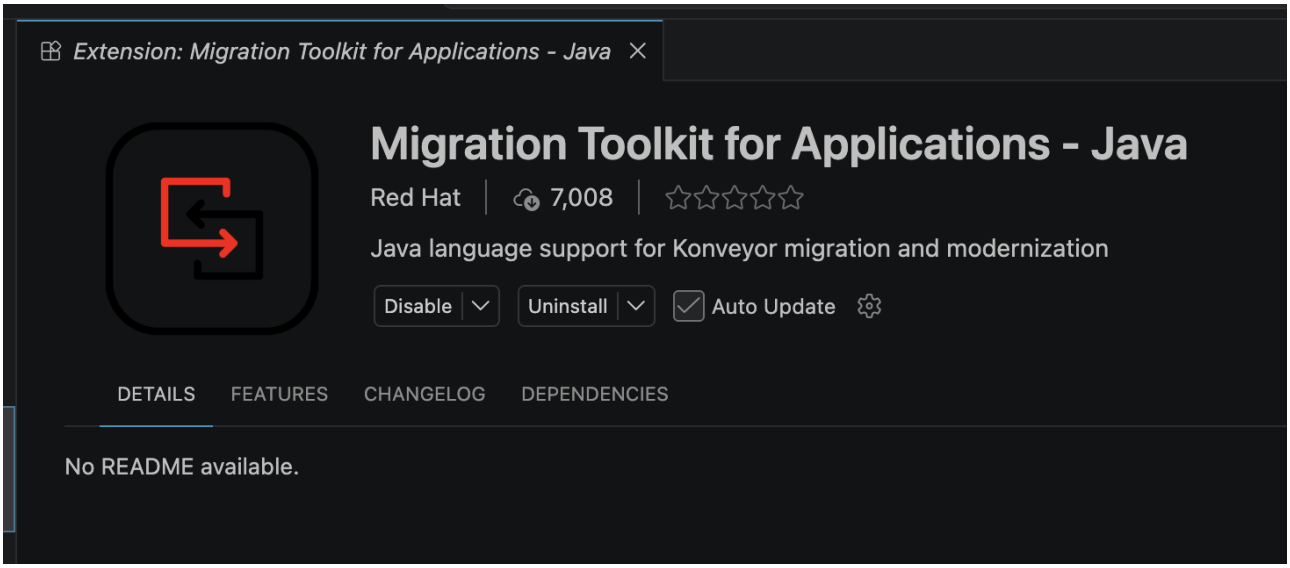


Figure 44. Migration Toolkit for Applications Extension(Click the image to view it in full size in a new browser tab)

2. Once the installation is complete, verify that the MTA extension icon appears in the left-hand Activity Bar. Select this icon to open the MTA Analysis view.

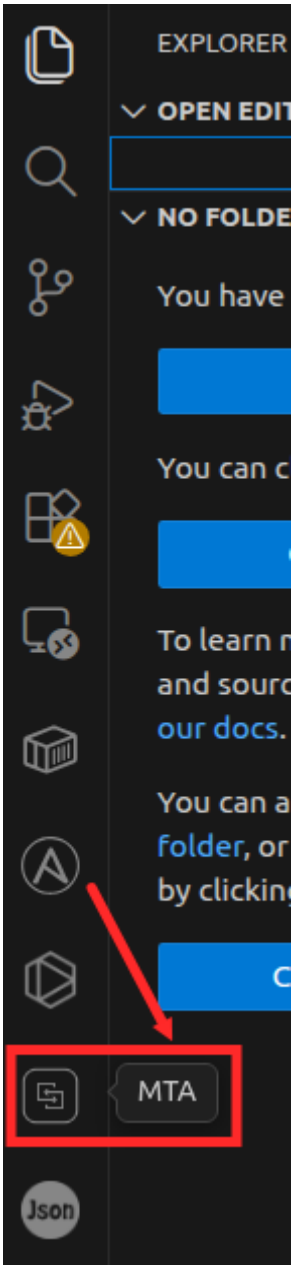


Figure 45. MTA Extension Icon(Click the image to view it in full size in a new browser tab)

3. Verify that the MTA Analysis view displays the following options:

   Open Analysis Panel Manage Profiles

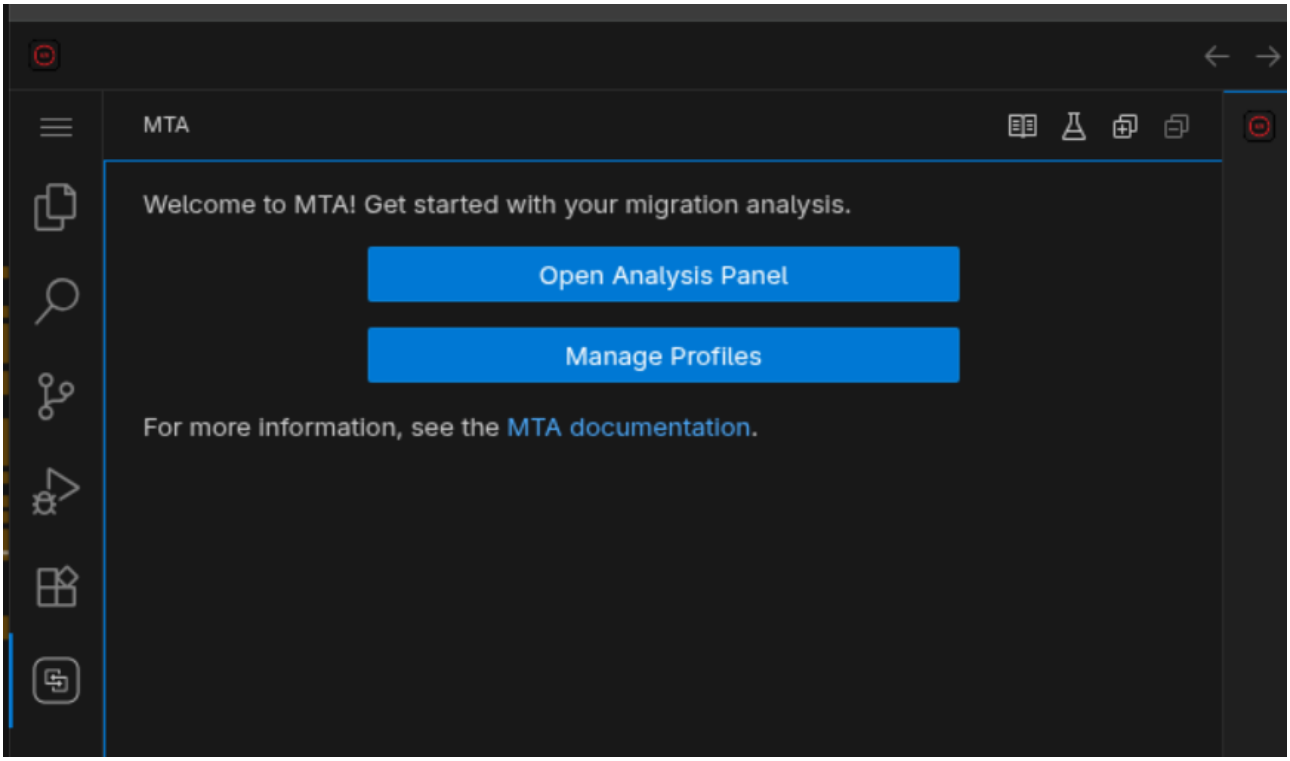


Figure 46. MTA Extension Installed(Click the image to view it in full size in a new browser tab)

Configure the MTA Extension

Connect the MTA extension to the MTA web console and create a migration profile for the Coolstore application.

1. Retrive the OPENSHIFT_CLUSTER_INGRESS_DOMAIN from below command as you will need it in upcoming steps.

```
OPENSHIFT_CLUSTER_INGRESS_DOMAIN=$(oc get ingresses.config.openshift.io cluster -o
jsonpath='{.spec.domain}')
echo ${OPENSHIFT_CLUSTER_INGRESS_DOMAIN}
```

2. Open the Analysis Panel by clicking **[Open Analysis Panel]**

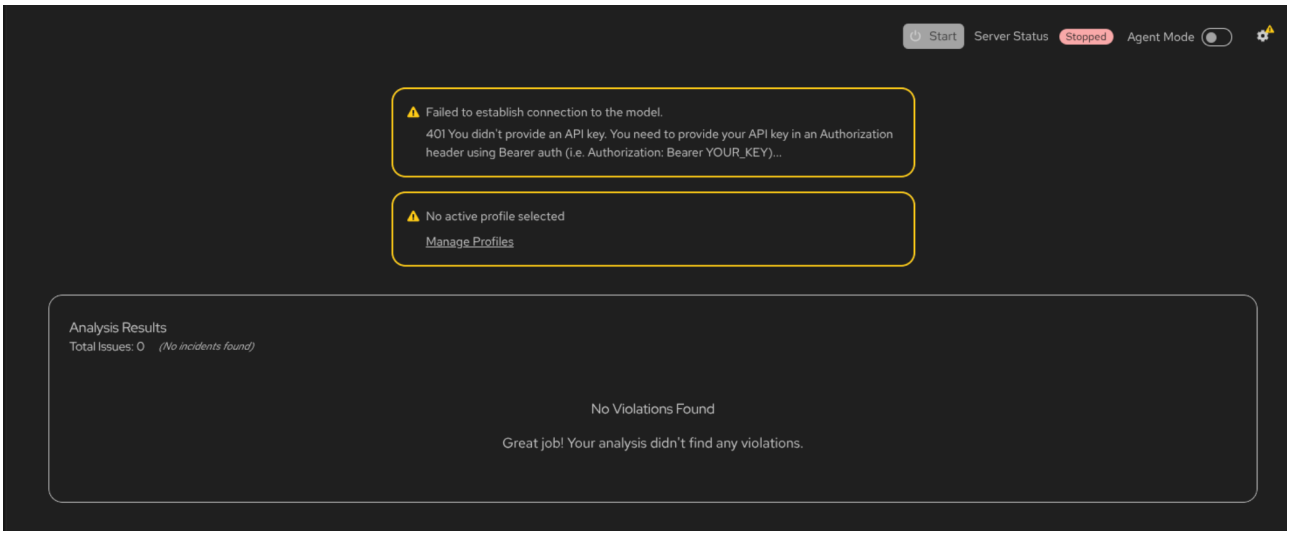


Figure 47. Analysis Panel(Click the image to view it in full size in a new browser tab)

3. From the top-right corner of the Visual Studio Code window, select the **Settings** option to access the MTA configuration panel.

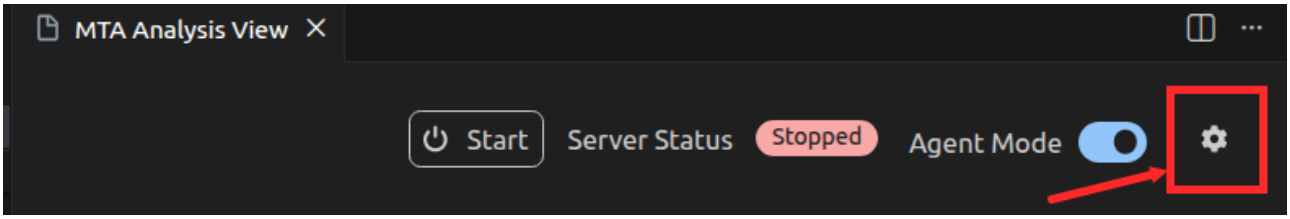


Figure 48. MTA Settings(Click the image to view it in full size in a new browser tab)

4. Click **[Configure Hub Settings]** under **Hub Configuration**

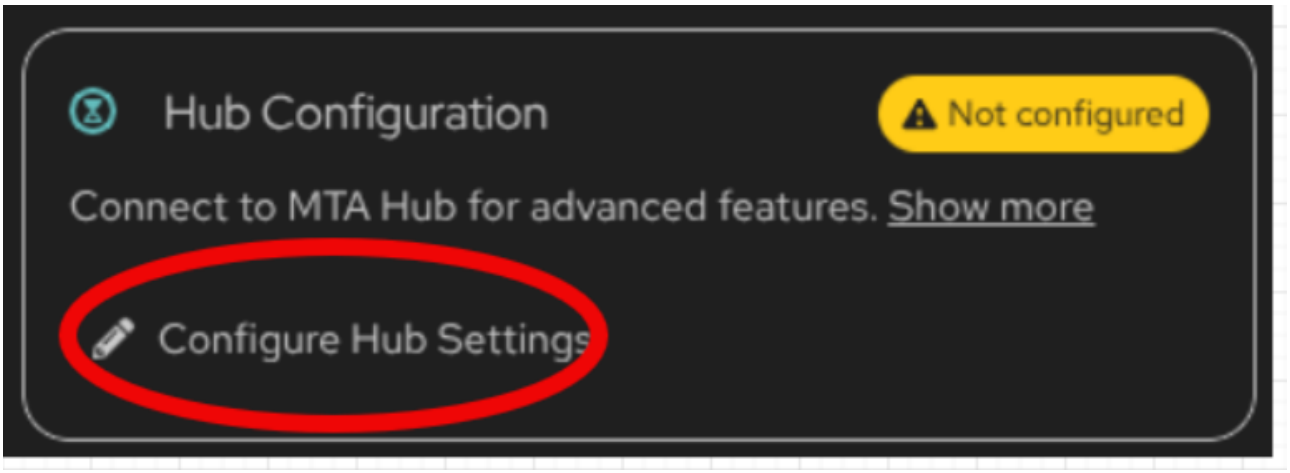


Figure 49. Hub Configuration(Click the image to view it in full size in a new browser tab)

In the settings screen, fill in the details, make sure to substitute **OPENSHIFT_CLUSTER_INGRESS_DOMAIN** you retreive from **Step 1**.

- **Enable Hub**: Checked
- **Hub URL**: `https://mta-openshift-mta.${OPENSHIFT_CLUSTER_INGRESS_DOMAIN}
- **Insecure connection**: Checked
- **Enable authentication**: Unchecked
- **Solution Server**: Enabled
- **Profile Sync**: Enabled

  Click **[Save]**

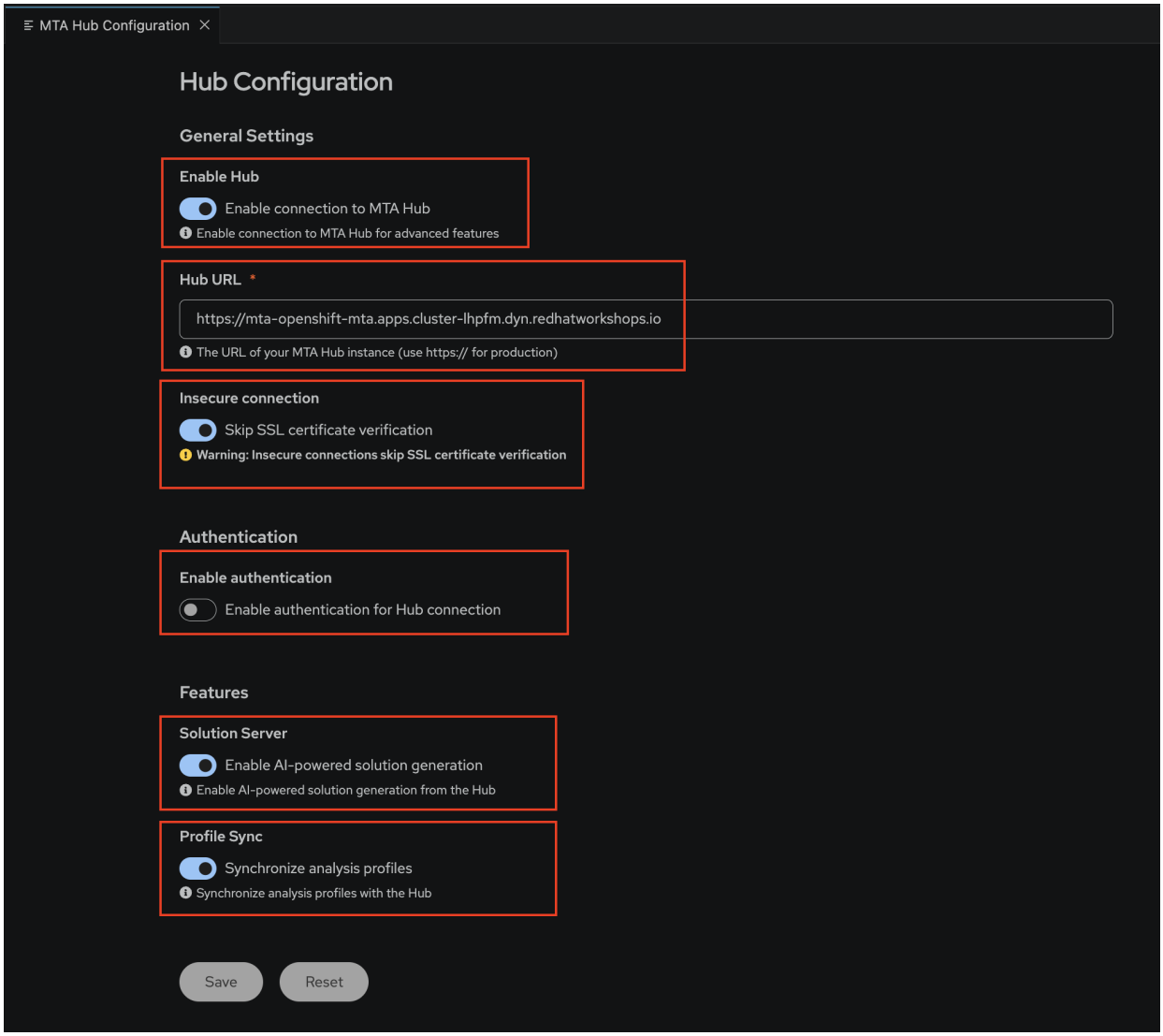


Figure 50. Hub Settings(Click the image to view it in full size in a new browser tab)

5. Close **MTA Hub Configuration** tab and open the **MTA Analysis Panel** tab.
6. Click **[Edit in Profile Manager]** to create a new profile

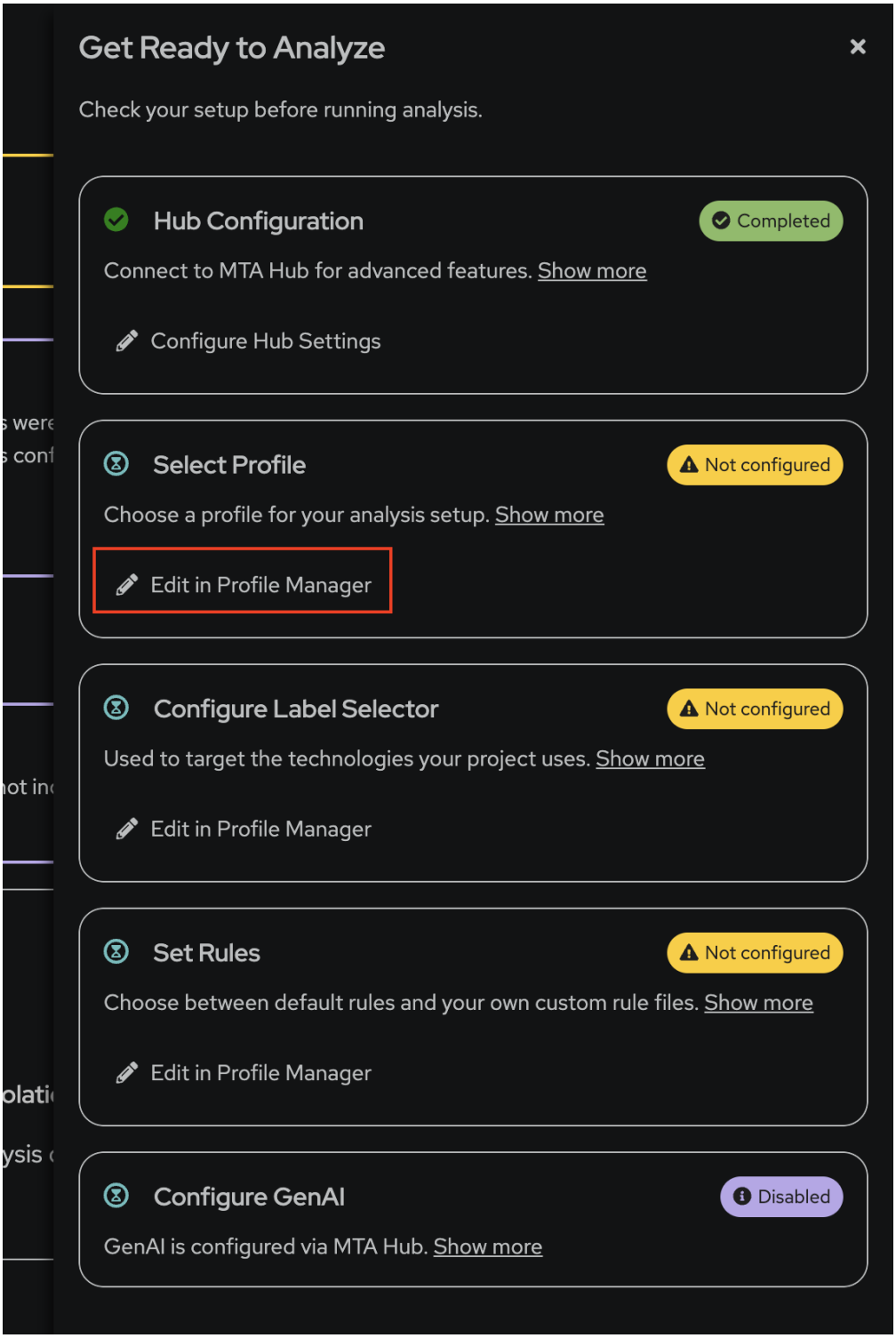


Figure 51. Edit in Profile Manager Settings(Click the image to view it in full size in a new browser tab)

7. Click **[New Profile]**

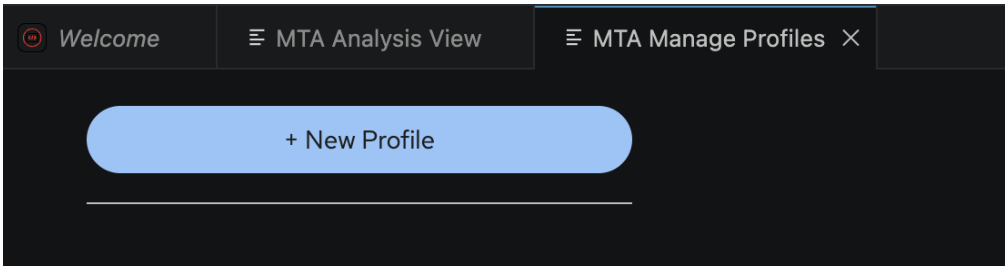


Figure 52. New Profile(Click the image to view it in full size in a new browser tab)

Fill in the details:

- **Profile Name**: `coolstore`
- **Target Technologies**: Select `eap8`
- **Source Technologies**: Select `eap7`
- **Use Default Rules**: Checked

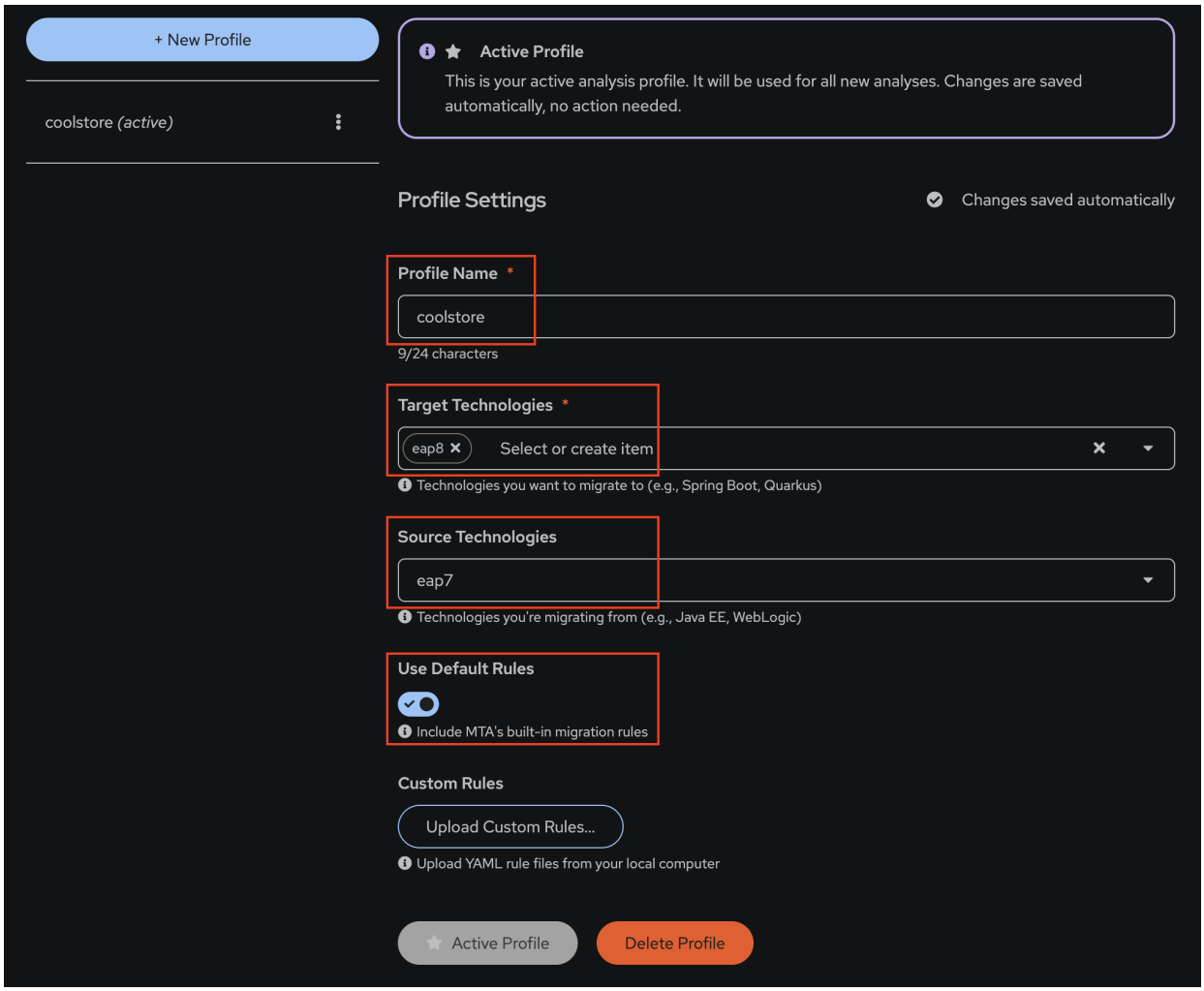


Figure 53. New Profile(Click the image to view it in full size in a new browser tab)

8. Final setting should look as below .Final Settings(*Click the image to view it in full size in a new browser tab*) image::final.png[align="center"]

Lab 8: Analyze and Modernize the Application

In this lab, you'll use the MTA extension with Developer Lightspeed and Agent Mode to analyze the Coolstore application and apply AI-assisted recommendations to migrate it from JBoss EAP 7 to JBoss EAP 8.

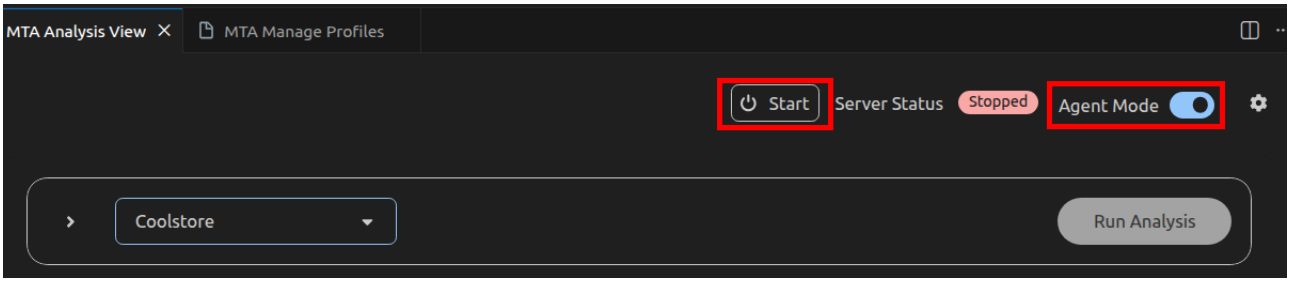
By the end of this lab, you will have:

- Analyzed the Coolstore application using the configured MTA profile
- Reviewed AI-assisted migration recommendations

- Applied recommended code changes using Developer Lightspeed
- Re-analyzed the application and verified the updated results
- Built the migrated application successfully

Analyze the Application

1. Return to the **MTA Analysis View**, confirm that **Agent Mode** is enabled, and select **[Start]** to activate the newly created profile.



2. Select **[Run Analysis]** to begin the application analysis. MTA, together with Developer Lightspeed and the configured GenAI model, analyzes the application source code. Wait until the analysis reaches **100% completion**.

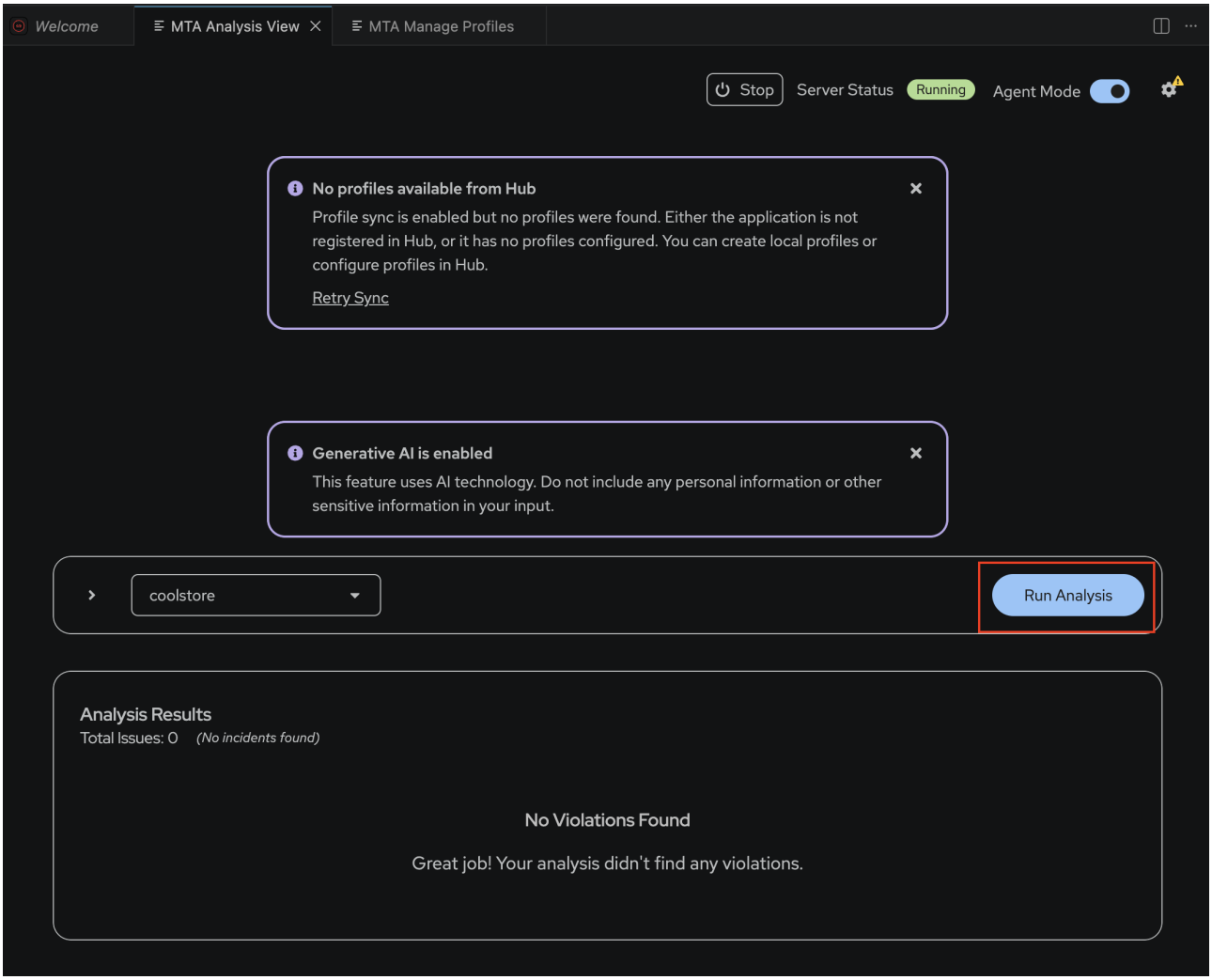


*Figure 54. Run Analysis(**Click the image to view it in full size in a new browser tab**)*

3. Review the analysis results. The results identify migration issues and provide guided

recommendations for migrating the application from EAP 7 to EAP 8.

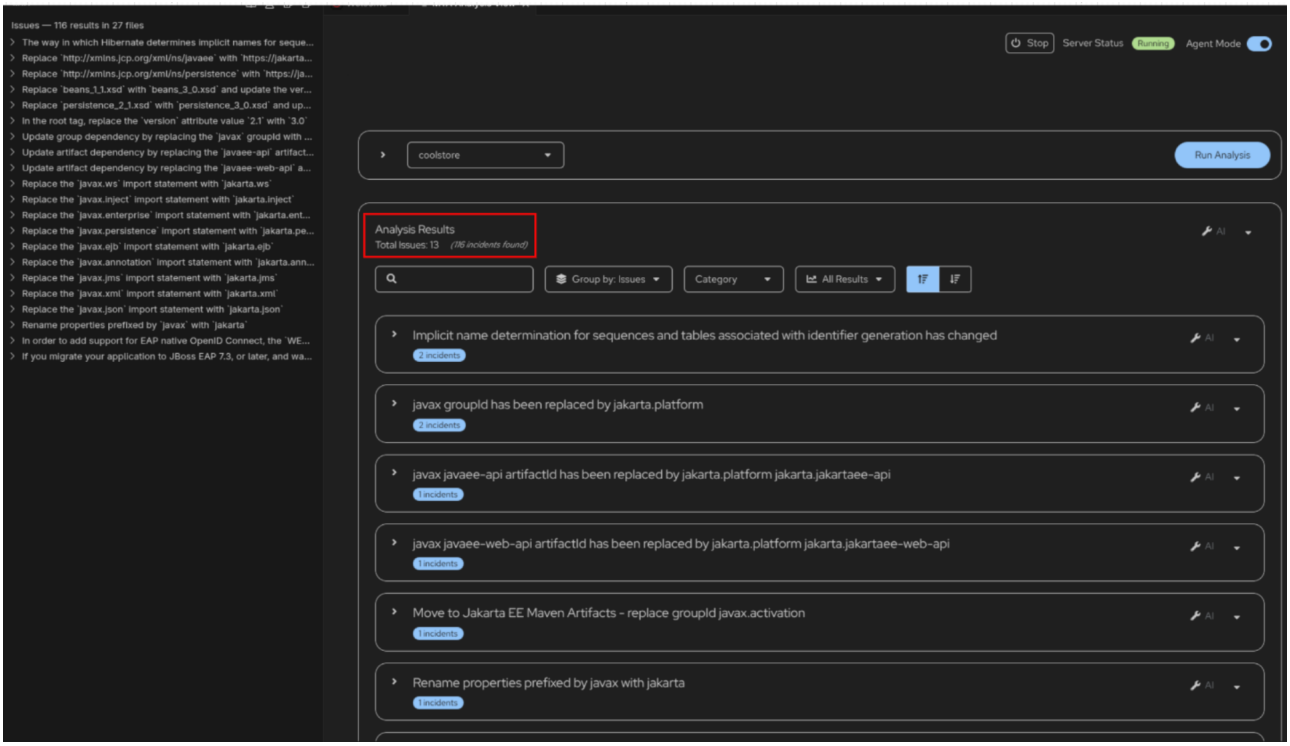


*Figure 55. Analysis Results(***Click the image to view it in full size in a new browser tab***)*

4. Select the reported issue `javax groupId has been replaced by jakarta.platform` and review the accompanying Arcade video. The video demonstrates how Developer Lightspeed proposes code changes directly within the source files.



Applying recommended changes can introduce additional impacts in other parts of the codebase. When **Agent Mode** is enabled, Developer Lightspeed can identify and suggest these follow-on changes. Without **Agent Mode**, these dependent updates may not be detected or reflected in the analysis.

1. Apply the recommended changes and repeat the analysis. Observe how the total number of reported issues decreases as issues are resolved, and review the updated analysis results.

Build the Migrated Application

1. Open the terminal and run the following command:

```
./mvn clean package
```



AI-generated changes may introduce errors in the `pom.xml` file. A separate repository (i.e `${GITLAB_URL}/mta-apps/coolstore-migrated.git`) containing the fully migrated application is available for comparison and verification. You can also use it to correct your implementation if required

Accepted solutions can be captured in the **Solution Server** of the MTA Hub and can be reused to generate code changes for future applications. This allows migration solutions to be applied

consistently across multiple applications.

Once the analysis is complete, all issues are resolved, and `./mvn clean package` succeeds, you have completed the application migration exercise.

Lab Summary

In this section, you used **Red Hat OpenShift Dev Spaces**, the **Migration Toolkit for Applications (MTA) extension**, and **Red Hat Developer Lightspeed** to modernize the Coolstore application from **JBoss EAP** 7 to **JBoss EAP 8**.

You first created and prepared a Dev Spaces workspace, installed the required Java language support and MTA extension, and connected the extension to the MTA web console. You then created a migration profile targeting JBoss EAP 8.

Next, you used **Developer Lightspeed** with **Agent Mode** to analyze the application, review AI-assisted migration recommendations, and apply the recommended code changes directly in the IDE. You re-ran the analysis to verify the migration issues were resolved and built the migrated application successfully.

By completing this section, you have:

- Set up a Dev Spaces workspace for the Coolstore application
- Installed and configured the MTA extension
- Connected Dev Spaces to the MTA web console
- Created an EAP 7 to EAP 8 migration profile
- Used Developer Lightspeed and Agent Mode for AI-assisted migration
- Applied and verified migration code changes
- Successfully built the modernized application

You have now completed the end-to-end application modernization workflow using **MTA, Developer Lightspeed, and OpenShift Dev Spaces**.

Knowledge Check

<iframe type="text/javascript" src="_attachments/questions.html" style="width: 100%; max-width: 1024px; height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>

Summary and What's next?

*Congratulations! You have successfully established the foundation of your **Application Modernization with MTA**.*

In this module, you learned how to modernize enterprise Java applications using Red Hat Migration Toolkit for Applications (MTA), Red Hat Developer Lightspeed, and Red Hat OpenShift Dev Spaces.

You explored MTA's approach to application modernization at scale, configured and deployed MTA, and analyzed the Coolstore application for migration from JBoss EAP 7 to JBoss EAP 8. You reviewed migration findings, including the **Java EE (javax) to Jakarta EE (jakarta)** changes required for the migration.

You then used the **MTA extension in Red Hat OpenShift Dev Spaces with Developer Lightspeed and Agent Mode to apply AI-assisted migration** recommendations directly in the IDE, re-analyze the application, and validate the modernized application.

In the next module, you'll explore how Trusted Software Supply Chain practices help secure applications, dependencies, artifacts, and software delivery pipelines throughout the Software Factory.